Universidad de las fuerzas Armadas "ESPE"

Departamento de Electrónica en Telecomunicaciones

Ingeniería en Telecomunicaciones

Resolución de Ejercicios Prácticos

Taller Práctico de Programación

Luis Mateo Ayala Guala

Fundamentos de Programación

Quito, Ecuador 26 de octubre de 2026

Índice

Contenido

Universidad de las fuerzas Armadas “ESPE” 1

Introducción 9

Desarrollo de los Ejercicios 10

1.1 Media de dos Valores 10

Análisis 10

Tabla de Objetos 10

Seudocódigo 10

Código 10

Diagrama de Flujo 11

Prueba de Escritorio 12

1.2 Valor absoluto de x al cubo 13

Análisis 13

Tabla de Objetos 13

Seudocódigo 13

Código 14

Diagrama de Flujo 15

Prueba de Escritorio 16

1.3 Divisible 17

Análisis 17

Tabla de Objetos 17

Seudocódigo 17

Código 17

Diagrama de Flujo 19

Prueba de Escritorio 20

1.4. Intervalo 21

Análisis 21

Tabla de Objetos 21

Seudocódigo 21

Código 21

Diagrama de Flujo 22

Prueba de Escritorio 22

1.5. Conversión de unidades de tiempo 24

Análisis 24

Tabla de Objetos 24

Seudocódigo 24

Código 25

Diagrama de flujo 26

Prueba de Escritorio 26

1.6. Máximo, mínimo y media de alturas 27

Análisis 27

Tabla de Objetos 27

Seudocodigo..... 28

Código..... 28

Diagrama de Flujo 30

Prueba de Escritorio..... 31

1.7. Raíz cuadrada entera..... 31

Análisis 31

Tabla de Objetos..... 31

Seudocódigo..... 32

Código..... 32

Diagrama de Flujo 33

Prueba de Escritorio..... 33

1.8. Número primo..... 34

Análisis 34

Tabla de Objetos..... 34

Seudocódigo..... 35

Código..... 35

Diagrama de Flujo 36

Prueba de Escritorio..... 37

1.9. Números primos entre dos números dados..... 38

Análisis 38

Tabla de Objetos..... 38

Seudocódigo..... 39

Código..... 39

Diagrama de Flujo 41

Prueba de Escritorio..... 42

1.10. Factorial 43

Análisis 43

Tabla de Objetos 43

Seudocódigo..... 43

Código..... 43

Diagrama de Flujo 45

Prueba de Escritorio..... 45

Para Pensar..... 46

1.11. Serie de Fibonacci..... 47

Análisis 47

Tabla de Objetos 47

Seudocódigo..... 47

Código..... 48

Diagrama de Flujo 49

Prueba de Escritorio..... 49

Para Pensar..... 50

1.12. Suma de dígitos..... 50

Análisis 50

Tabla de Objetos 50

Seudocódigo..... 51

Código..... 51

Diagrama de Flujo 52

Prueba de Escritorio..... 53

Para Pensar..... 53

1.13. Número capicúa 54

Análisis 54

Tabla de objetos..... 54

Seudocódigo..... 54

Código..... 55

Diagrama de Flujo 56

Prueba de Escritorio..... 57

Para Pensar..... 57

1.14. Máximo común divisor (algoritmo de Euclides) 58

Análisis 58

Tabla de Objetos..... 58

Seudocódigo..... 58

Código..... 59

Diagrama de Flujo 60

Prueba de Escritorio..... 61

Para Pensar..... 61

1.15. Año bisiesto 62

Análisis 62

Tabla de Objetos 62

Seudocódigo 62

Código 63

Diagrama de Flujo 64

Prueba de Escritorio 64

Para Pensar 66

1.16. Número perfecto 66

Análisis 66

Tabla de Objetos 66

Seudocódigo 67

Código 67

Diagrama de Flujo 69

Prueba de Escritorio 70

Para Pensar 71

1.17. Conversión de decimal a binario 71

Análisis 71

Tabla de Objetos 71

Seudocódigo 72

Código 72

Diagrama de Flujo 74

Prueba de Escritorio 75

Para Pensar..... 76

1.18. Triángulo de asteriscos 76

Análisis 76

Tabla de Objetos 76

Seudocódigo..... 77

Código..... 77

Diagrama de Flujo 79

Prueba de Escritorio..... 80

Para Pensar..... 81

1.19. Segundo mayor de una secuencia 81

Análisis 81

Tabla de Objetos 82

Seudocódigo..... 82

Código..... 83

Diagrama de Flujo 84

Prueba de Escritorio..... 85

Para Pensar..... 85

1.20. Adivina el número 86

Análisis 86

Tabla de Objetos 86

Seudocódigo..... 86

Código..... 87

Diagrama de Flujo 88

Prueba de Escritorio..... 89

Para Pensar..... 89

Rubrica de Pensamiento Critico 90

Puntaje total: 90

Introducción

El siguiente trabajo recopila la resolución de distintos problemas planteados que incluyen distintos desafíos de algoritmia y programación, cada problema contiene su

respectivo análisis, tabla de objetos, seudocódigo, diagrama de flujo, prueba de escritorio y su respectivo codigo.

Desarrollo de los Ejercicios

1.1 Media de dos Valores

Análisis

Entada: Dos numeros al azar.

Proceso: Se suman los 2 valores y los dividimos para 2.

Salida: El calculo total de la media.

Tabla de Objetos

Objeto	**Nombre en C**	**Tipo de Dato**	**Valor Inicial**
Primer Número	num1	Float	0
Segundo Número	num2	Float	0
Resultado de la Media	media	Float	0

Seudocódigo

```
Algoritmo Problema_1_1
   Definir num1, num2, media Como Real
   Escribir "Ingrese el primer numero:"
   Leer num1
   Escribir "Ingrese el segundo numero:"
   Leer num2
   media <- (num1 + num2) / 2.0
   Escribir "La media es: ", media
FinAlgoritmo
```

Código

```
#include <stdio.h>
```

```
int main() {
    float num1, num2, media;

    printf("Ingrese el primer numero: ");
    scanf("%f", &num1);

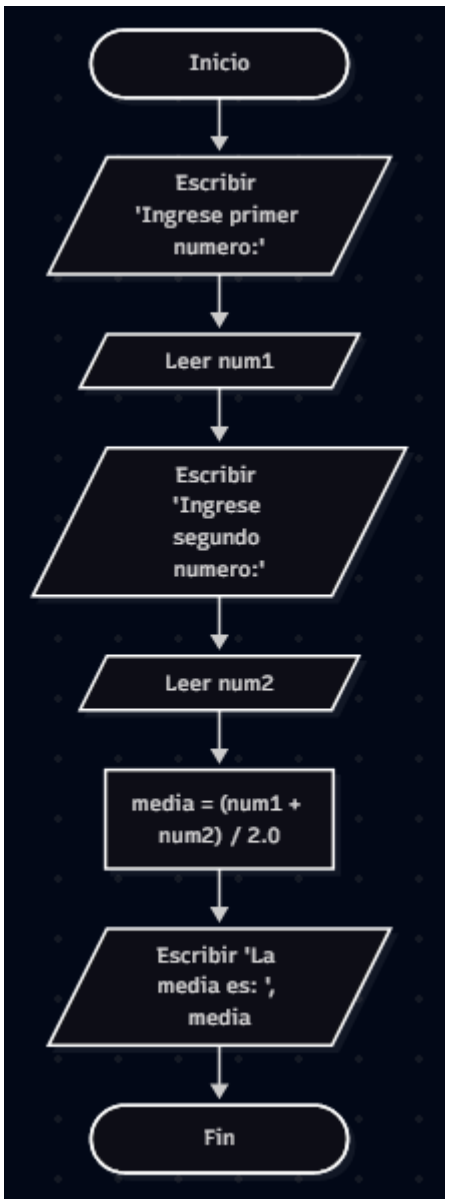
    printf("Ingrese el segundo numero: ");
    scanf("%f", &num2);

    media = (num1 + num2) / 2.0;

    printf("La media es: %.2f\n", media);

    return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio

Start here | *perfect.c

```c
//Ejercicio 1.1 Media de 2 Valores
//Realizado por Luis Ayala
#include <stdio.h>

int main() {
    float num1, num2, media;

    printf("Ingrese el primer numero: ");
    scanf("%f", &num1);

    printf("Ingrese el segundo numero: ");
    scanf("%f", &num2);

    media = (num1 + num2) / 2.0;

    printf("La media es: %.2f\n", media);

    return 0;
}
```

```
C:\Users\LUIS\OneDrive\Documentos\catesis...
Ingrese el primer numero:
```

Start here | *perfect.c

```c
//Ejercicio 1.1 Media de 2 Valores
//Realizado por Luis Ayala
#include <stdio.h>

int main() {
    float num1, num2, media;

    printf("Ingrese el primer numero: ");
    scanf("%f", &num1);

    printf("Ingrese el segundo numero: ");
    scanf("%f", &num2);

    media = (num1 + num2) / 2.0;

    printf("La media es: %.2f\n", media);

    return 0;
}
```

```
C:\Users\LUIS\OneDrive\Documentos\catesis...
Ingrese el primer numero: 10
Ingrese el segundo numero: 20
La media es: 15.00

Process returned 0 (0x0)   execution time : 286.753 s
Press any key to continue.
```

Start here | *perfect.c

```c
//Ejercicio 1.1 Media de 2 Valores
//Realizado por Luis Ayala
#include <stdio.h>

int main() {
    float num1, num2, media;

    printf("Ingrese el primer numero: ");
    scanf("%f", &num1);

    printf("Ingrese el segundo numero: ");
    scanf("%f", &num2);

    media = (num1 + num2) / 2.0;

    printf("La media es: %.2f\n", media);

    return 0;
}
```

```
C:\Users\LUIS\OneDrive\Documentos\cates...
Ingrese el segundo numero: 8.5
La media es: 8.00

Process returned 0 (0x0)   execution time : 4.467 s
Press any key to continue.
```

Nº	num1	num2	media	Salida
1	10	20	15	La media es:15
2	7.5	8.5	8	La media es: 8

1.2 Valor absoluto de x al cubo

Análisis

Entrada: Un numero al azar.

Proceso: Para sacar el valor absoluto si el numero ingresado es menor que 0, se multiplica por -1, luego se multiplica por si mismo 3 veces para elevarlo al cubo.

Salida: El valor absoluto elevado al cubo.

Tabla de Objetos

Objeto	Nombre	Tipo de Dato	Valor Inicial
Numero Ingresado	x	float	0
Valor Absoluto	abs_x	float	0
Resultado al Cubo	cubo	float	0

Seudocódigo

```
Algoritmo Problema_1_2
   Definir x, abs_x, cubo Como Real
   Escribir "Ingrese un numero:"
   Leer x
   Si x < 0 Entonces
      abs_x <- x * -1.0
   SiNo
      abs_x <- x
   FinSi
   cubo <- abs_x * abs_x * abs_x
   Escribir "El valor absoluto al cubo es: ", cubo
FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
   float x, abs_x, cubo;

   printf("Ingrese un numero: ");
   scanf("%f", &x);

   if (x < 0) {
      abs_x = x * -1.0;
   } else {
      abs_x = x;
   }

   cubo = abs_x * abs_x * abs_x;

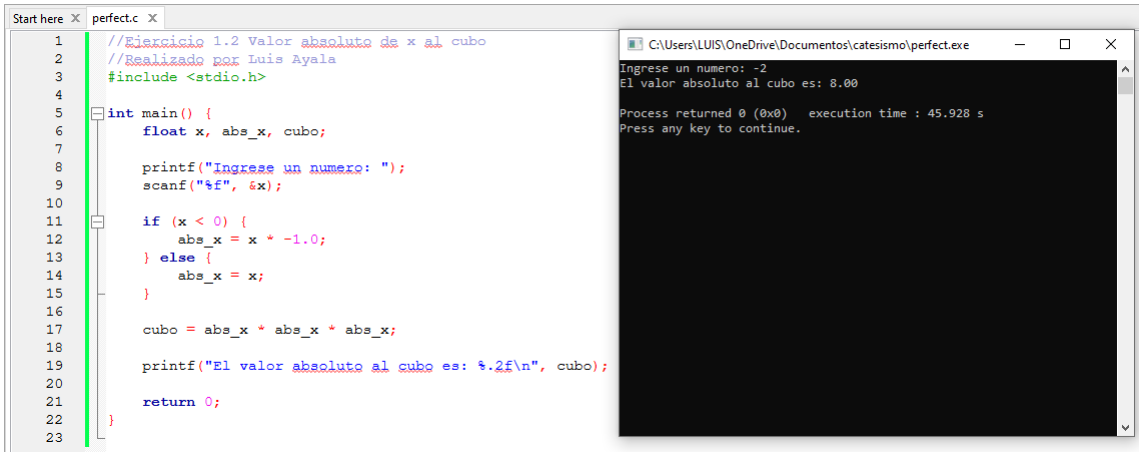
   printf("El valor absoluto al cubo es: %.2f\n", cubo);

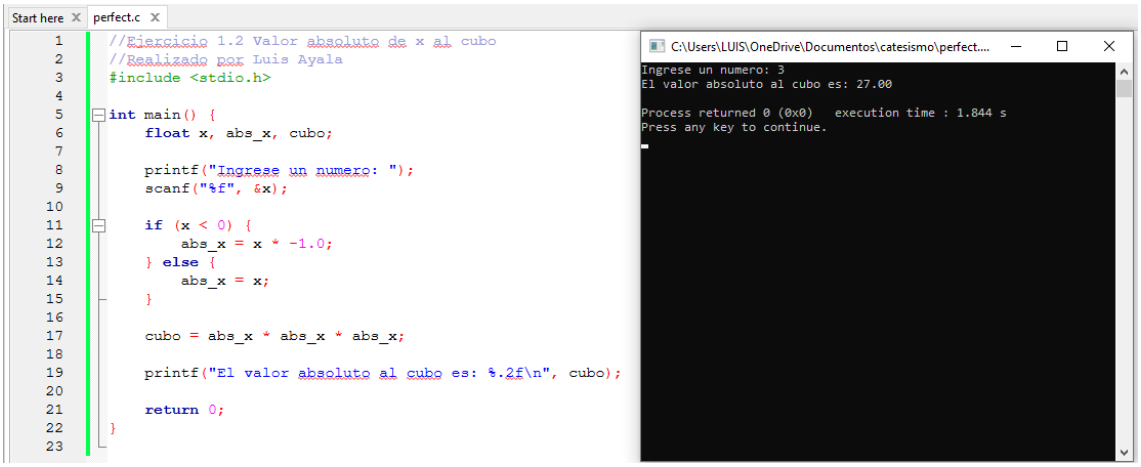
   return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio





x	x<0	abs_x	cubo	Salida
-2	V	2	8	El valor absoluto al cubo es: 8
3	F	3	27	El valor absoluto al cubo es: 27

1.3 Divisible

Análisis

Entrada: Dos numeros Enteros

Proceso: Se usa el operador (%) para obtener el residuo de la división, si el resultado es 0, el primer numero es divisible por el segundo.

Salida: Un mensaje indicando si es o no es divisible.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Primer Entero	num1	Int	0
Segundo Entero	num2	Int	0

Seudocódigo

```
Algoritmo Problema_1_3
   Definir num1, num2 Como Entero
   Escribir "Ingrese el primer numero entero:"
   Leer num1
   Escribir "Ingrese el segundo numero entero:"
   Leer num2
   Si num2 <> 0 Entonces
      Si num1 MOD num2 = 0 Entonces
         Escribir "El numero SI es divisible."
      SiNo
         Escribir "El numero NO es divisible."
      FinSi
   SiNo
      Escribir "Error: No se puede dividir para cero."
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

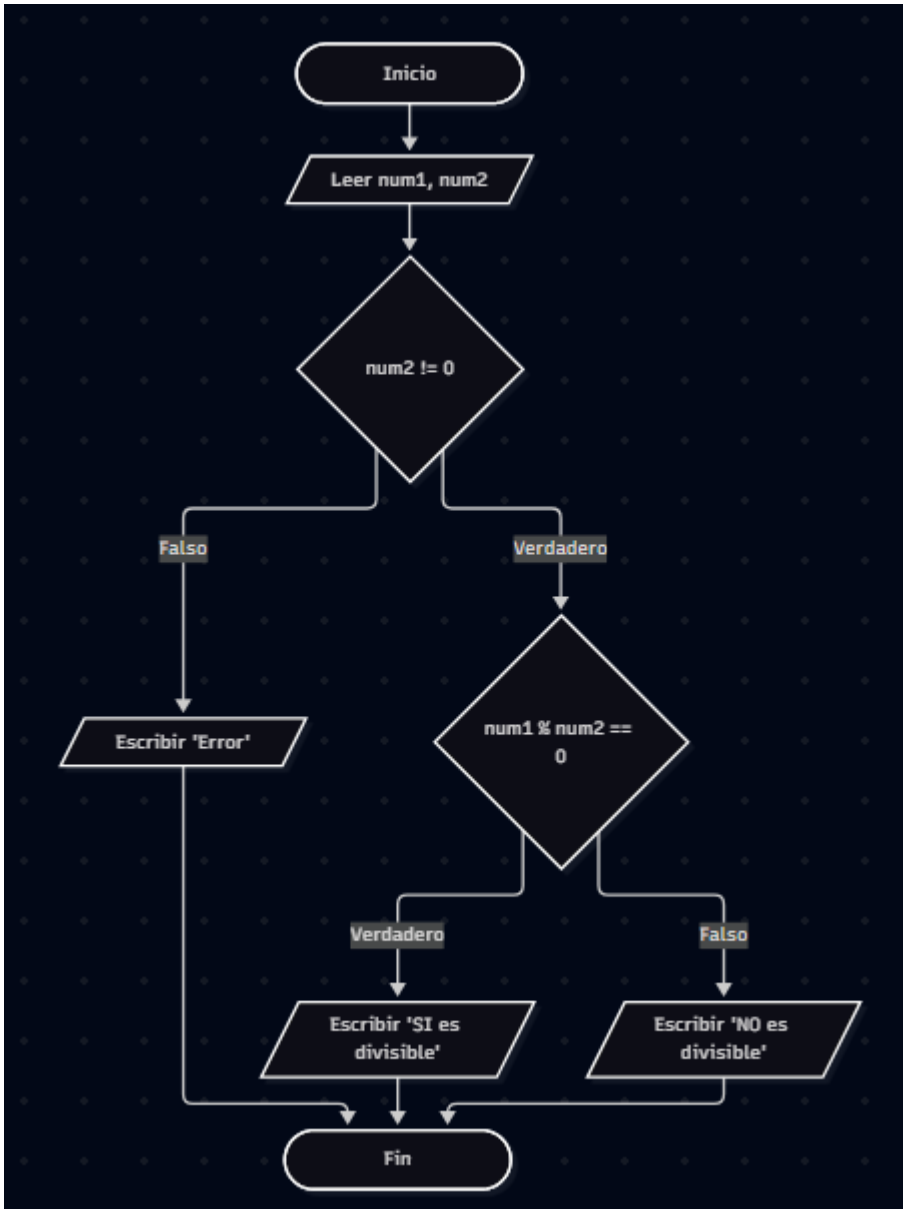
int main() {
   int num1, num2;
```

```
    printf("Ingrese el primer numero entero: ");
    scanf("%d", &num1);
    printf("Ingrese el segundo numero entero: ");
    scanf("%d", &num2);

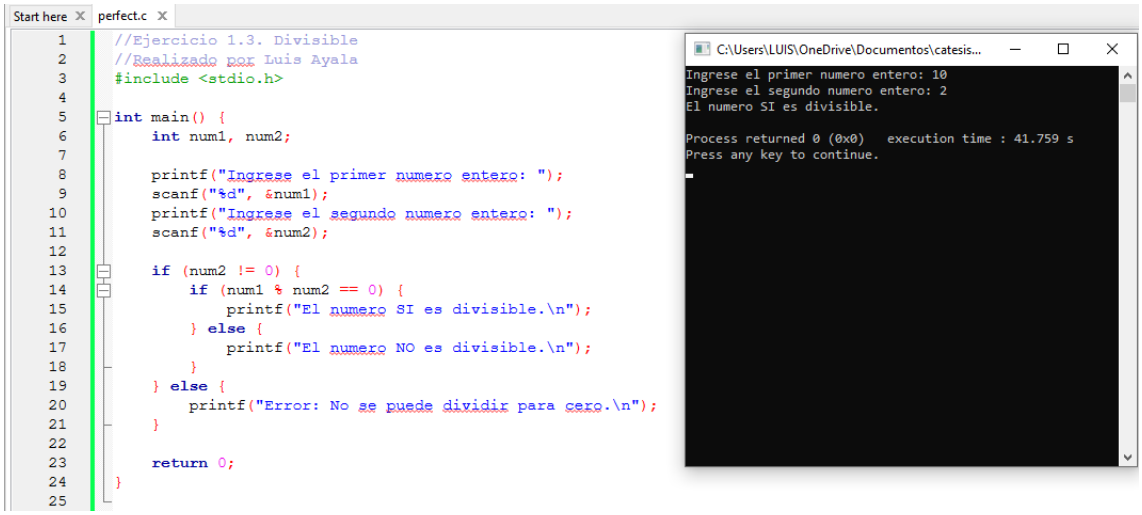
    if (num2 != 0) {
        if (num1 % num2 == 0) {
            printf("El numero SI es divisible.\n");
        } else {
            printf("El numero NO es divisible.\n");
        }
    } else {
        printf("Error: No se puede dividir para cero.\n");
    }

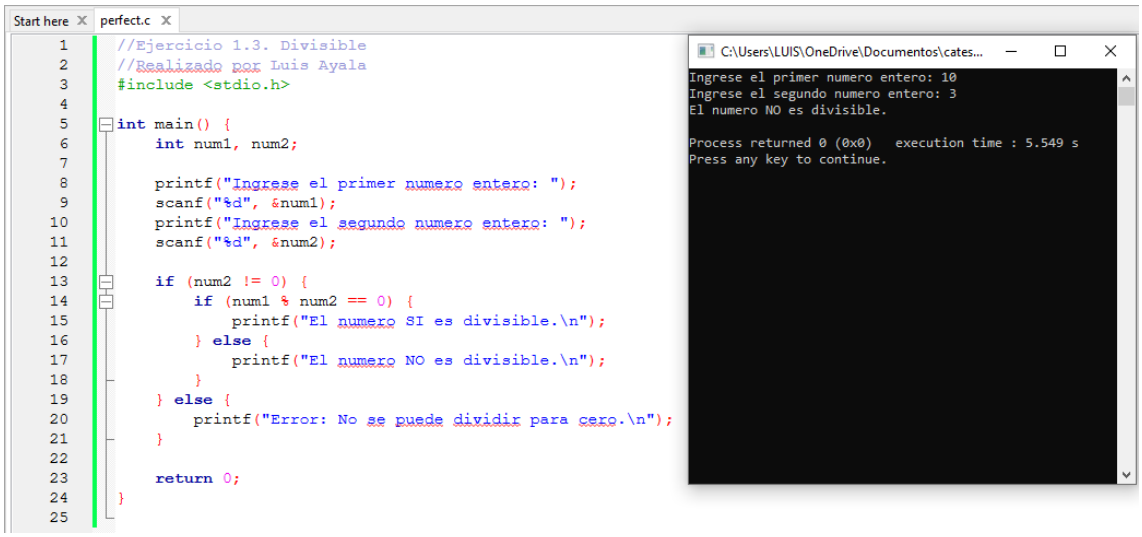
    return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio





num1	num2	num2 !=0	num1%num2==0	Salida
10	2	V	V	El numero SI es divisible
10	3	V	F	El numero NO es divisible

1.4. Intervalo

Análisis

Entrada: Cualquier Número.

Proceso: Verificamos mediante (&&) si el numero cumple ambas condiciones a la ves; ser mayor o iguala 0, y menor o igual a 10.

Salida: Confirmación de si pertenece al intervalo, o no.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Numero a evaluar	x	float	0

Seudocódigo

```
Algoritmo Problema_1_4
   Definir x Como Real
   Escribir "Ingrese un numero:"
   Leer x
   Si x >= 0 Y x <= 10 Entonces
      Escribir "El numero pertenece al intervalo [0, 10]."
   SiNo
      Escribir "El numero NO pertenece al intervalo [0, 10]."
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

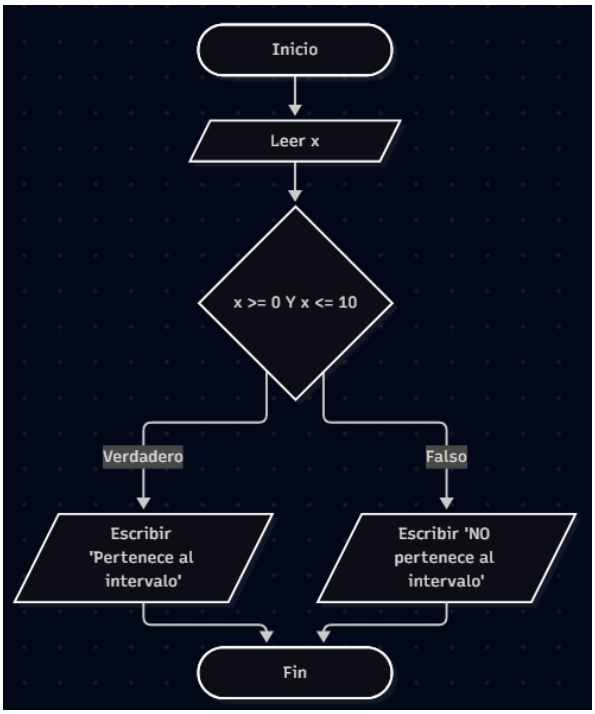
int main() {
   float x;

   printf("Ingrese un numero: ");
   scanf("%f", &x);

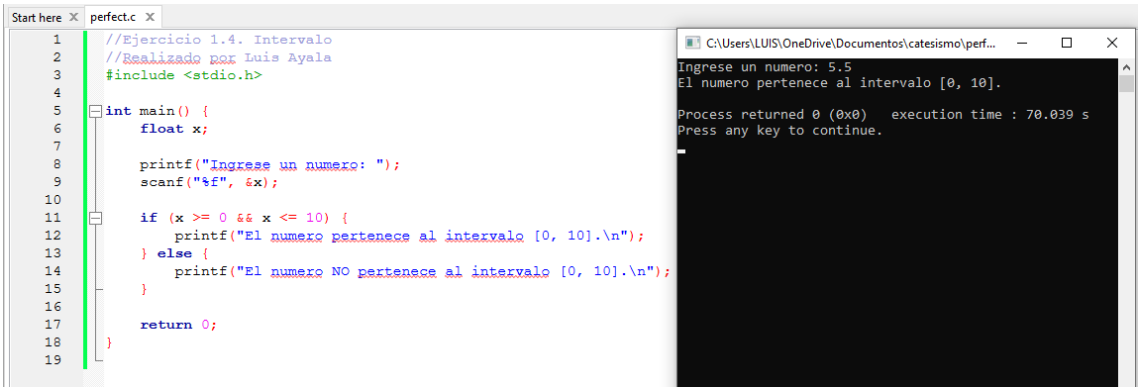
   if (x >= 0 && x <= 10) {
      printf("El numero pertenece al intervalo [0, 10].\n");
   } else {
      printf("El numero NO pertenece al intervalo [0, 10].\n");
   }
```

```
    return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



```
//Ejercicio 1.4. Intervalo
//Realizado por Luis Ayala
#include <stdio.h>

int main() {
    float x;

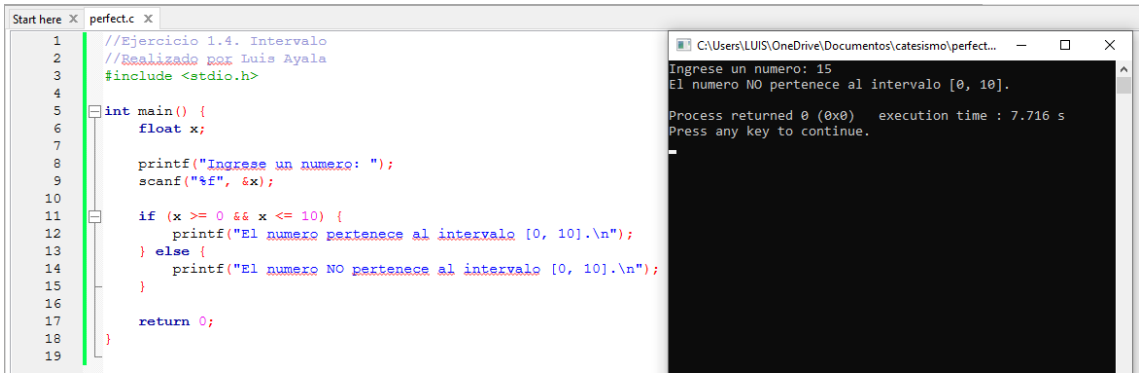
    printf("Ingrese un numero: ");
    scanf("%f", &x);

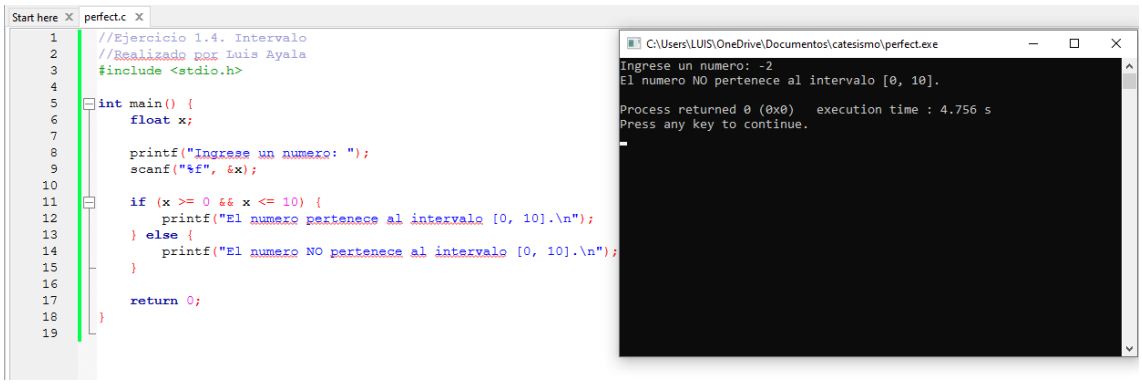
    if (x >= 0 && x <= 10) {
        printf("El numero pertenece al intervalo [0, 10].\n");
    } else {
        printf("El numero NO pertenece al intervalo [0, 10].\n");
    }

    return 0;
}
```

```
C:\Users\LUIS\OneDrive\Documentos\catesismo\perf...
Ingrese un numero: 5.5
El numero pertenece al intervalo [0, 10].

Process returned 0 (0x0)   execution time : 70.039 s
Press any key to continue.
```





x	**x>=0**	**x<=0**	**x >= 0 && x <= 10**	**Salida**
5.5	V	V	V	El numero pertenece al intervalo [0, 10].
15	V	F	F	El numero NO pertenece al intervalo [0, 10].
-2	F	V	F	El numero NO pertenece al

				intervalo [0, 10].

1.5. Conversión de unidades de tiempo

Análisis

Entrada: Un valor entero que representara una cantidad total de segundos.

Proceso: Se utiliza la división entera para el calculo de días, horas y minutos completos; tras cada división se usa el operador (%) para actualizar la variable que designemos con los segundos sobrantes que no alcanzan a formar una unidad superior.

Salida: Cantidad de días, horas, minutos y segundos.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Segundos Totales	c	Int	0
Dias	días	Int	0
Horas	horas	Int	0
Minutos	minutos	Int	0
Segundos Finales	segundos	Int	0

Seudocódigo

```
Algoritmo Problema_1_5
   Definir c, dias, horas, minutos, segundos Como Entero
   Escribir "Ingrese la cantidad de segundos:"
   Leer c
   dias <- TRUNC(c / 86400)
   c <- c MOD 86400
   horas <- TRUNC(c / 3600)
   c <- c MOD 3600
   minutos <- TRUNC(c / 60)
```

```
        segundos <- c MOD 60
        Escribir "Equivale a: ", dias, " dias, ", horas, " horas, ", minutos, " minutos y ", segundos, " segundos."
    FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
    int c, dias, horas, minutos, segundos;

    printf("Ingrese la cantidad de segundos: ");
    scanf("%d", &c);

    dias = c / (24 * 60 * 60);
    c = c % (24 * 60 * 60);

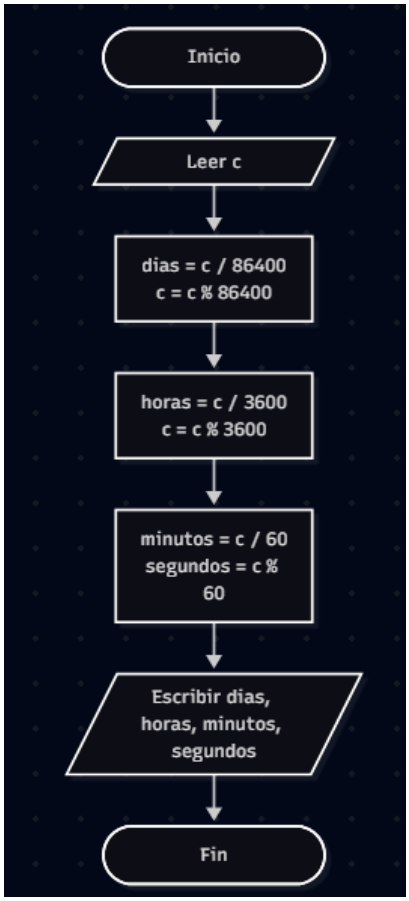
    horas = c / (60 * 60);
    c = c % (60 * 60);

    minutos = c / 60;
    segundos = c % 60;

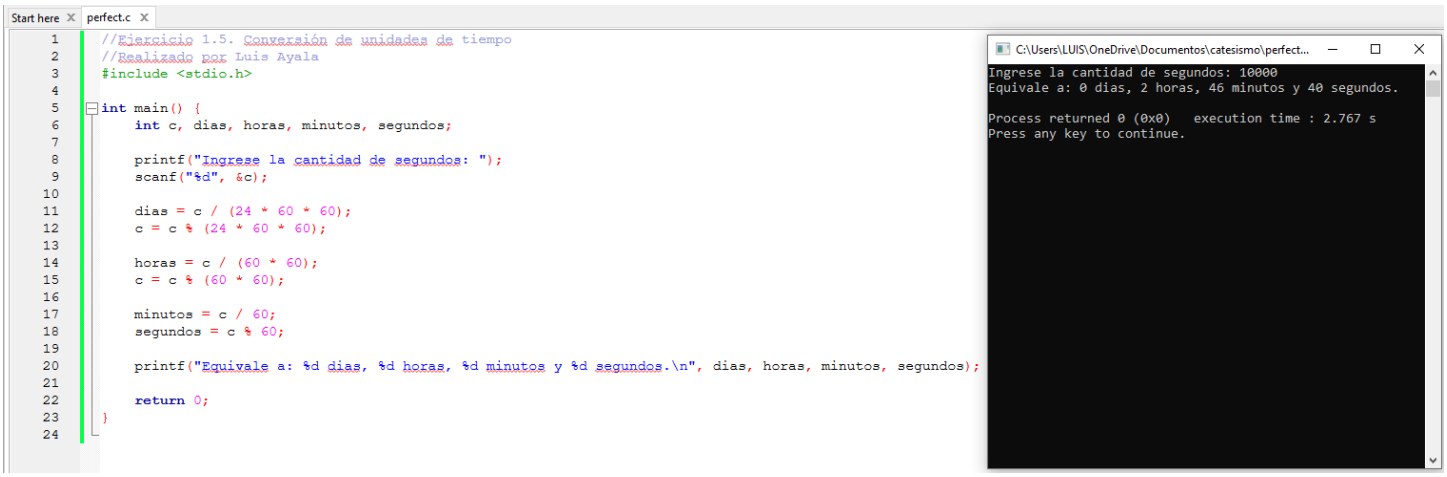
    printf("Equivale a: %d dias, %d horas, %d minutos y %d segundos.\n", dias, horas, minutos, segundos);

    return 0;
}
```

Diagrama de flujo



Prueba de Escritorio



Start here | perfect.c

```
1  //Ejercicio 1.5. Conversión de unidades de tiempo
2  //Realizado por Luis Ayala
3  #include <stdio.h>
4
5  int main() {
6      int c, dias, horas, minutos, segundos;
7
8      printf("Ingrese la cantidad de segundos: ");
9      scanf("%d", &c);
10
11     dias = c / (24 * 60 * 60);
12     c = c % (24 * 60 * 60);
13
14     horas = c / (60 * 60);
15     c = c % (60 * 60);
16
17     minutos = c / 60;
18     segundos = c % 60;
19
20     printf("Equivale a: %d dias, %d horas, %d minutos y %d segundos.\n", dias, horas, minutos, segundos);
21
22     return 0;
23 }
24
```

C:\Users\LUIS\OneDrive\Documentos\catesismo\perfect...

```
Ingrese la cantidad de segundos: 10000
Equivale a: 0 dias, 2 horas, 46 minutos y 40 segundos.

Process returned 0 (0x0)   execution time : 2.767 s
Press any key to continue.
```

Instrucción	Operación	Valor Resultante	Variable Actualizada
dias = c / 86400	10000 / 86400	0	dias = 0
c = c % 86400	10000 % 86400	10000	c = 10000
horas = c / 3600	10000 / 3600	2	horas = 2
c = c % 3600	10000 % 3600	2800	c = 2800
minutos = c / 60	2800 / 60	46	minutos = 46
segundos = c % 60	2800 % 60	40	segundos = 40

1.6. Máximo, mínimo y media de alturas

Análisis

Entrada: Un número indeterminado de estaturas, deteniendo el ciclo cuando el usuario ingresa un valor negativo.

Proceso: Se necesita un bucle que evalúe si le estatura es mayor o igual a 0, dentro de este bulce se suma la altura a un acumulador que cuenta cuantas van y compara si la altura actual es mayor al máximo registrado o menor al mínimo.

Salida: La media Calculada, además de la Altura máxima y mínima.

Tabla de Objetos

Objeto	Nombre	Tipo de Dato	Valor Inicial
Altura Ingresada	altura	Float	0
Suma Total	suma	Float	0
Cantidad de Alturas	cant	Int	0

Altura Máxima	max	Float	0
Altura Mínima	min	Float	0
Media Calculada	media	Float	0

Seudocodigo

```
Algoritmo Problema_1_6
   Definir altura, suma, max, min, media Como Real
   Definir cant Como Entero
   suma <- 0.0
   cant <- 0
   Escribir "Ingrese una altura (negativo para salir):"
   Leer altura
   Si altura >= 0 Entonces
      max <- altura
      min <- altura
   FinSi
   Mientras altura >= 0 Hacer
      suma <- suma + altura
      cant <- cant + 1
      Si altura > max Entonces
         max <- altura
      FinSi
      Si altura < min Entonces
         min <- altura
      FinSi
      Escribir "Ingrese una altura (negativo para salir):"
      Leer altura
   FinMientras
   Si cant > 0 Entonces
      media <- suma / cant
      Escribir "Media de alturas: ", media
      Escribir "Altura maxima: ", max
      Escribir "Altura minima: ", min
   SiNo
      Escribir "No se ingresaron alturas validas."
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
   float altura, suma = 0.0, max, min, media;
   int cant = 0;
```

```
    printf("Ingrese una altura (negativo para salir): ");
    scanf("%f", &altura);

    if (altura >= 0) {
        max = altura;
        min = altura;
    }

    while (altura >= 0) {
        suma = suma + altura;
        cant = cant + 1;

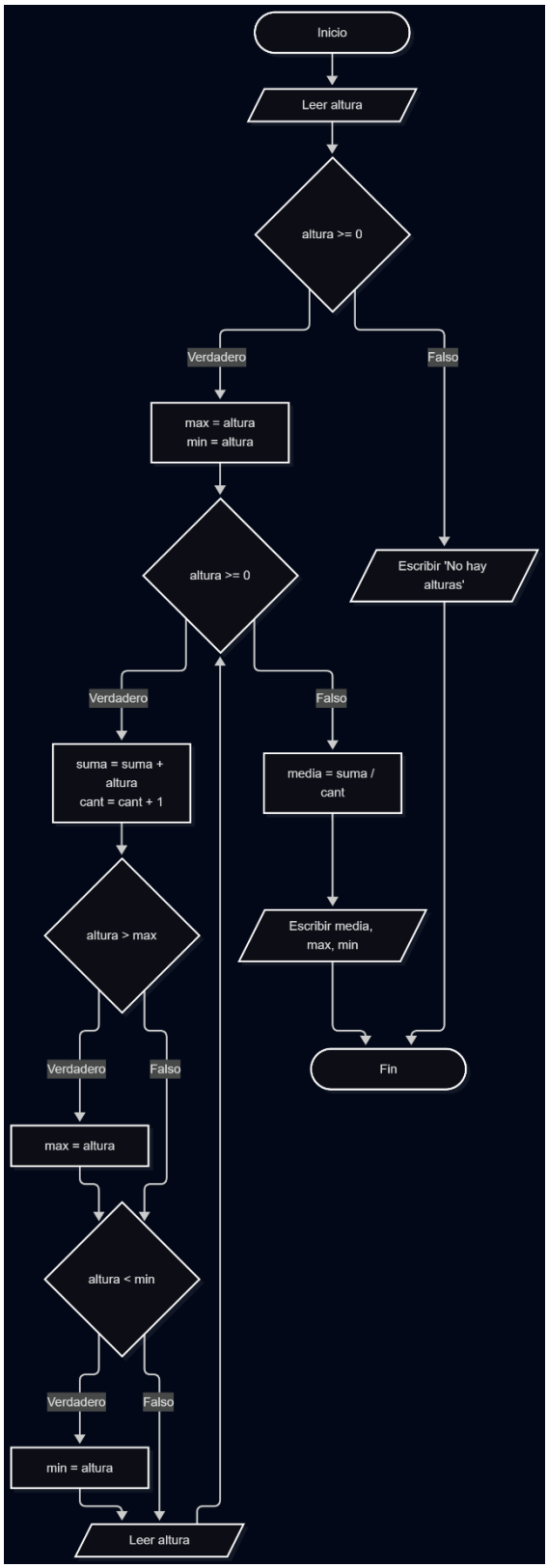
        if (altura > max) {
            max = altura;
        }
        if (altura < min) {
            min = altura;
        }

        printf("Ingrese una altura (negativo para salir): ");
        scanf("%f", &altura);
    }

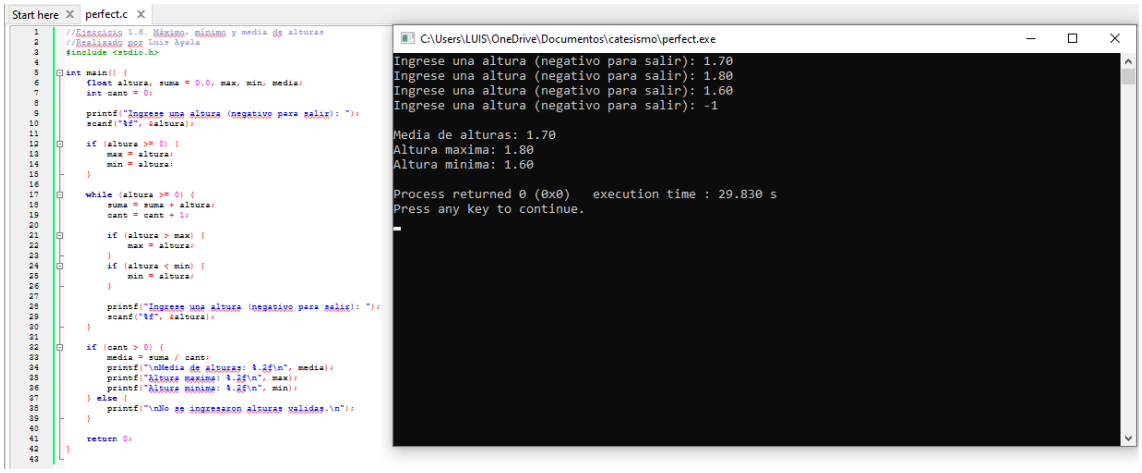
    if (cant > 0) {
        media = suma / cant;
        printf("\nMedia de alturas: %.2f\n", media);
        printf("Altura maxima: %.2f\n", max);
        printf("Altura minima: %.2f\n", min);
    } else {
        printf("\nNo se ingresaron alturas validas.\n");
    }

    return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



altura	cant	suma	max	min	Condicion
1.7	1	1.7	1.7	1.7	V
1.8	2	3.5	1.8	1.7	V
1.6	3	5.1	1.8	1.6	V
-1	3	5.1	1.8	1.6	F

1.7. Raíz cuadrada entera

Análisis

Entrada: Un numero Entero Positivo

Proceso: Incrementar un variable de uno en uno mientras si cuadrado sea menor o igual a el numero ingresado, al pasarse, la raíz entera será el valor interior.

Salida: La raíz cuadrada entera por defecto.

Tabla de Objetos

Objeto	Nombre	Tipo de Dato	Valor Inicial
Número ingresado	N	Int	0
Iterador/Prueba	i	Int	1
Raíz entera	R	Int	0

Seudocódigo

```
Algoritmo Problema_1_7
   Definir N, i, R Como Entero
   i <- 1
   Escribir "Ingrese un numero entero positivo:"
   Leer N
   Si N >= 0 Entonces
      Mientras i * i <= N Hacer
         i <- i + 1
      FinMientras
      R <- i - 1
      Escribir "La raiz cuadrada entera de ", N, " es: ", R
   SiNo
      Escribir "Error: Ingrese un numero positivo."
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
   int N, i = 1, R;

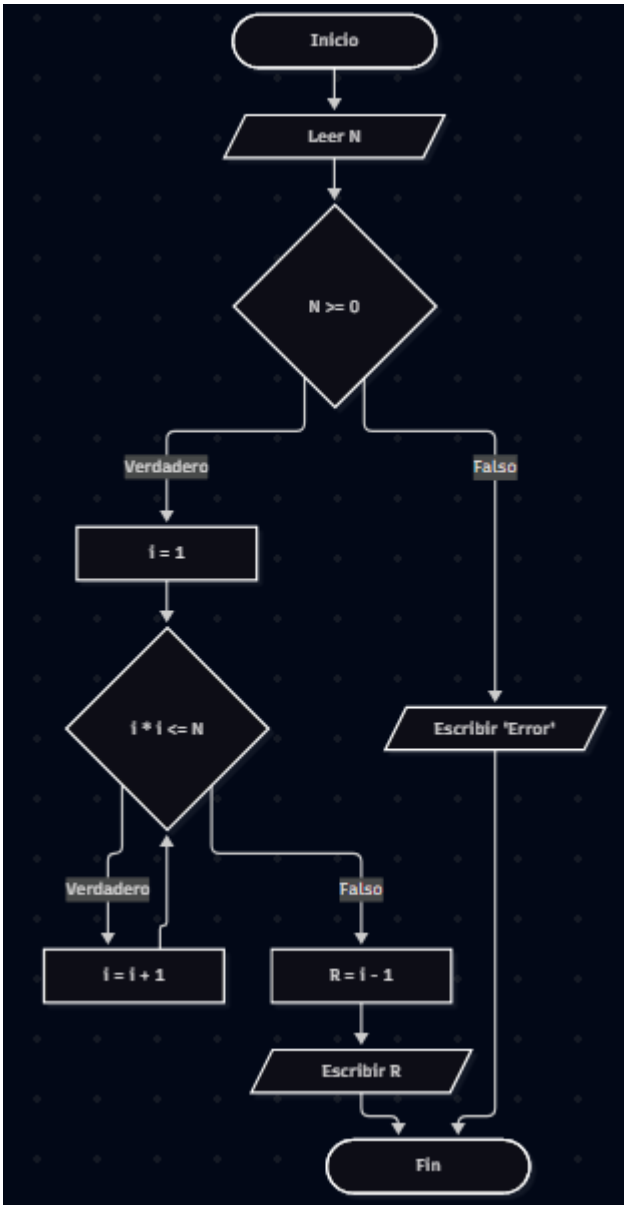
   printf("Ingrese un numero entero positivo: ");
   scanf("%d", &N);

   if (N >= 0) {
      while (i * i <= N) {
         i = i + 1;
      }

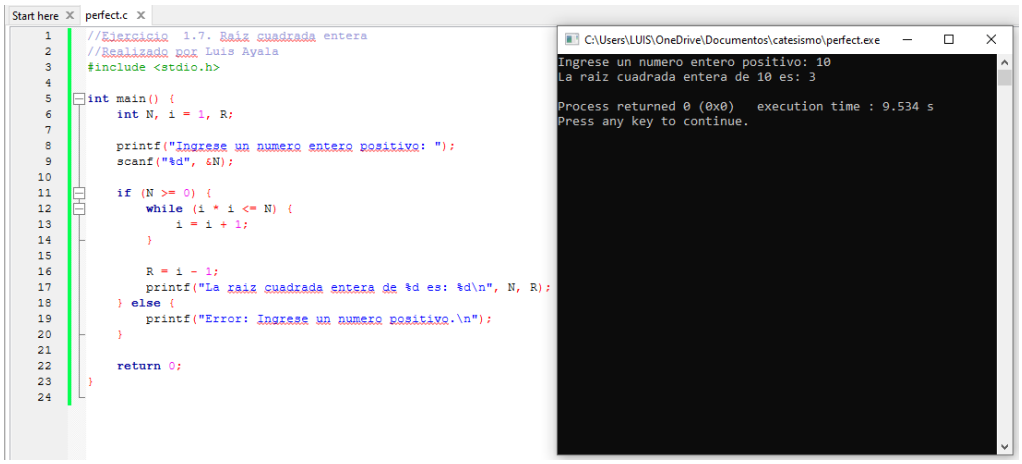
      R = i - 1;
      printf("La raiz cuadrada entera de %d es: %d\n", N, R);
   } else {
      printf("Error: Ingrese un numero positivo.\n");
   }

   return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



Start here | perfect.c

```
1  //Ejercicio  1.7. Raiz cuadrada entera
2  //Realizado por Luis Ayala
3  #include <stdio.h>
4
5  int main() {
6      int N, i = 1, R;
7
8      printf("Ingrese un numero entero positivo: ");
9      scanf("%d", &N);
10
11     if (N >= 0) {
12         while (i * i <= N) {
13             i = i + 1;
14         }
15
16         R = i - 1;
17         printf("La raiz cuadrada entera de %d es: %d\n", N, R);
18     } else {
19         printf("Error: Ingrese un numero positivo.\n");
20     }
21
22     return 0;
23 }
24
```

C:\Users\LUIS\OneDrive\Documentos\catesismo\perfect.exe

```
Ingrese un numero entero positivo: 10
La raiz cuadrada entera de 10 es: 3

Process returned 0 (0x0)   execution time : 9.534 s
Press any key to continue.
```

Iteración	**i**	**i*i**	**Condición** **i*i<=10**
1	1	1	V
2	2	4	V
3	3	9	V
4	4	16	F

1.8. Número primo

Análisis

Validar si un numero mayor a 1 solo es divisible para 1 y para si mismo, buscando divisores con un bucle.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Numero ingresado	num	Int	0
Divisor	i	Int	2
Bandera (1 si es primo, 0 si no)	esPrimo	Int	1

Seudocódigo

```
Algoritmo Problema_1_8
   Definir num, i, esPrimo Como Entero
   i <- 2
   esPrimo <- 1
   Escribir "Ingrese un numero entero mayor que 1:"
   Leer num
   Si num > 1 Entonces
      Mientras (i * i <= num) Y (esPrimo = 1) Hacer
         Si num MOD i = 0 Entonces
            esPrimo <- 0
         SiNo
            i <- i + 1
         FinSi
      FinMientras
      Si esPrimo = 1 Entonces
         Escribir "El numero ", num, " SI es primo."
      SiNo
         Escribir "El numero ", num, " NO es primo."
      FinSi
   SiNo
      Escribir "El numero debe ser mayor que 1."
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
   int num, i = 2, esPrimo = 1;

   printf("Ingrese un numero entero mayor que 1: ");
   scanf("%d", &num);

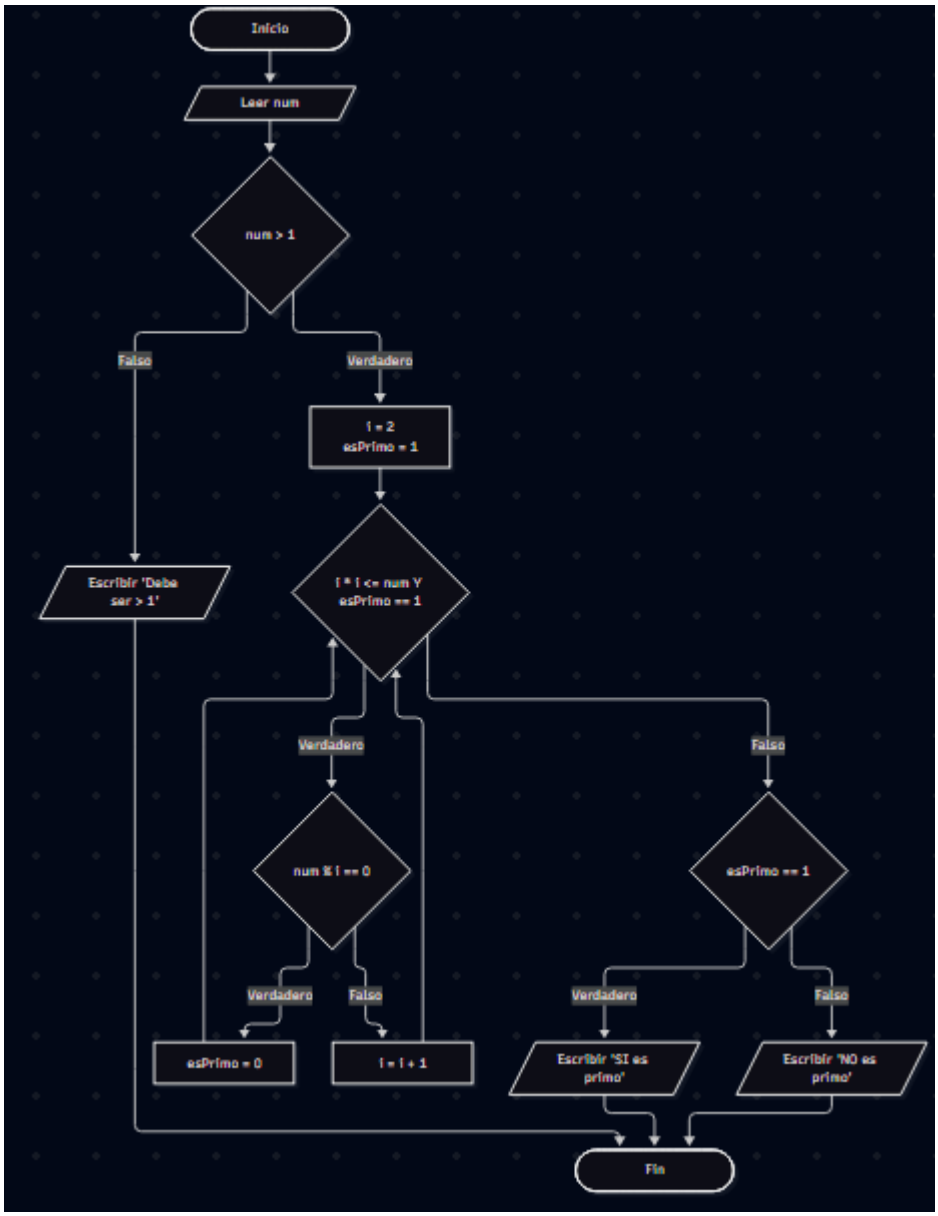
   if (num > 1) {
      while (i * i <= num && esPrimo == 1) {
         if (num % i == 0) {
            esPrimo = 0;
         } else {
            i = i + 1;
         }
      }

      if (esPrimo == 1) {
         printf("El %d SI es primo.\n", num);
      } else {
         printf("El %d NO es primo.\n", num);
      }
```

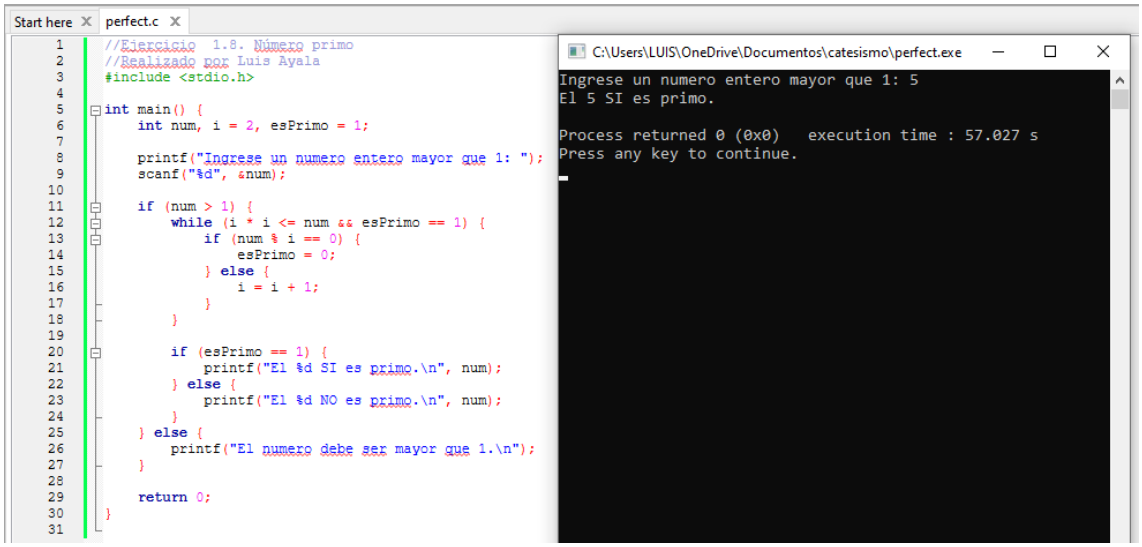
```
    } else {
        printf("El numero debe ser mayor que 1.\n");
    }

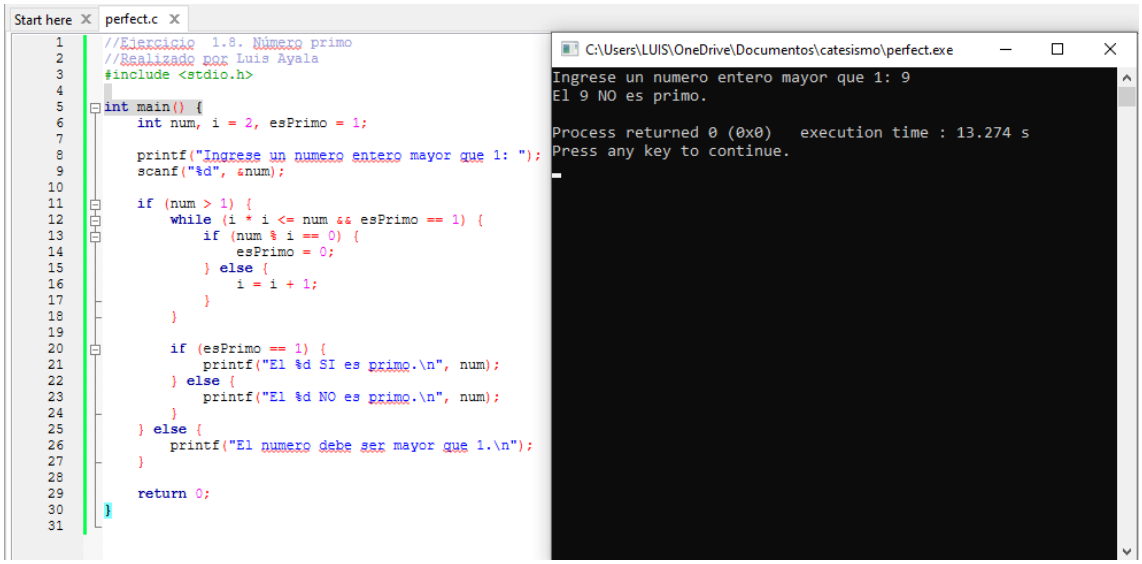
    return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio





Caso	**num**	**i**	**esPrimo**	**Condición while**	**Condición if**	**Acción Interna**	**Salida**
Prueba 1	5	2	1	4 <= 5 Y 1 == 1 (Verdadero)	5 % 2 == 0 **(Falso)**	i = 2 + 1 = 3	

	5	3	1	9 <= 5 (Falso- Sale)			El 5 SI es primo.
Prueba 2	9	2	1	4 <= 9 Y 1 == 1 (Verdadero)	i = 2 + 1 = 3		
	9	3	1	9 <= 9 Y 1 == 1 (Verdadero)	esPrimo = 0		
	9	3	0	1 == 1 (Falso - Sale)			El 9 NO es primo.

1.9. Números primos entre dos números dados

Análisis

Entrada: Dos numero enteros que marcan un inicio y fin del intervalo.

Proceso: Un bucle externo recorre cada numero del intervalo, verificamos si el número especifico es primo.

Salida: Todos los números primos encontrados en pantalla.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Limite Inferior	limInf	Int	0
Limite Superior	limSup	Int	0

Numero a Evaluar	num	Int	0
Divisor	i	Int	2
Bandera Primo	esPrimo	Int	1

Seudocódigo

```
Algoritmo Problema_1_9
   Definir limInf, limSup, num, i, esPrimo Como Entero
   Escribir "Ingrese el limite inferior:"
   Leer limInf
   Escribir "Ingrese el limite superior:"
   Leer limSup
   num <- limInf
   Mientras num <= limSup Hacer
      Si num > 1 Entonces
         esPrimo <- 1
         i <- 2
         Mientras (i * i <= num) Y (esPrimo = 1) Hacer
            Si num MOD i = 0 Entonces
               esPrimo <- 0
            SiNo
               i <- i + 1
            FinSi
         FinMientras
         Si esPrimo = 1 Entonces
            Escribir num
         FinSi
      FinSi
      num <- num + 1
   FinMientras
FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
   int limInf, limSup, num, i, esPrimo;

   printf("Ingrese el limite inferior: ");
   scanf("%d", &limInf);
   printf("Ingrese el limite superior: ");
   scanf("%d", &limSup);

   printf("Numeros primos entre %d y %d:\n", limInf, limSup);
```

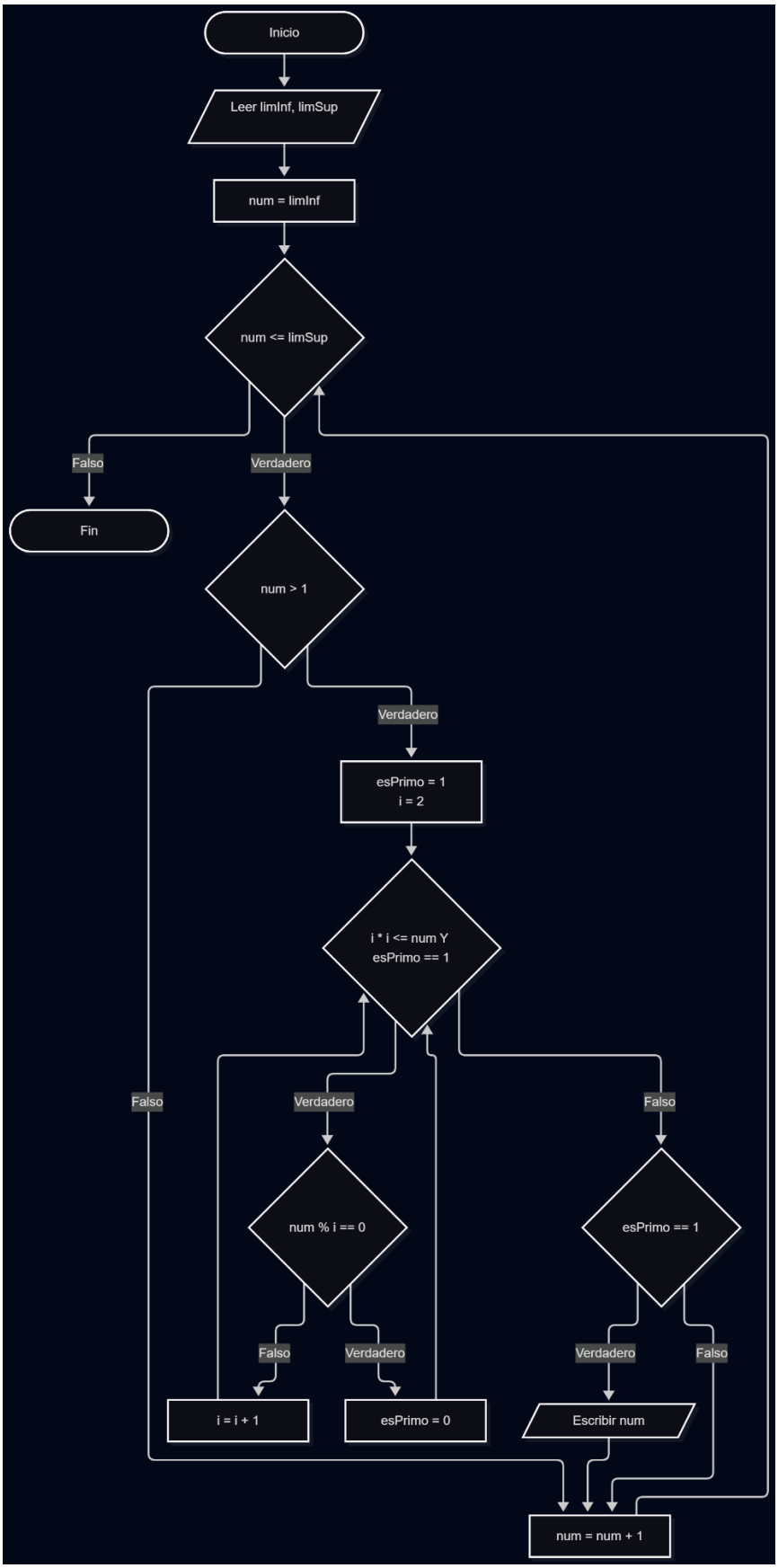
```
    num = limInf;
    while (num <= limSup) {
        if (num > 1) {
            esPrimo = 1;
            i = 2;

            while (i * i <= num && esPrimo == 1) {
                if (num % i == 0) {
                    esPrimo = 0;
                } else {
                    i = i + 1;
                }
            }

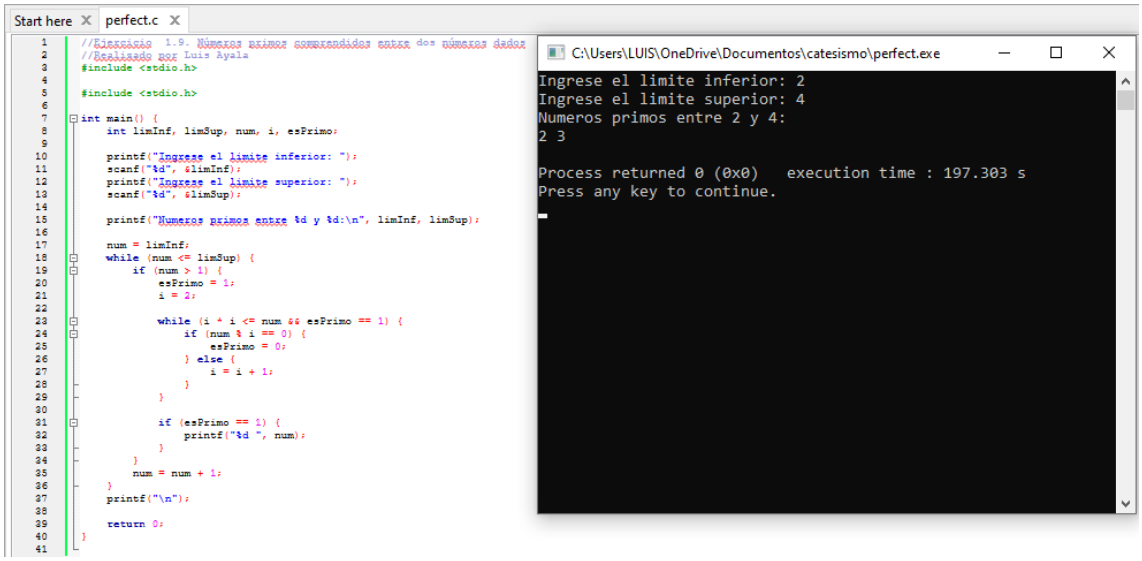
            if (esPrimo == 1) {
                printf("%d ", num);
            }
        }
        num = num + 1;
    }
    printf("\n");

    return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



num	num <= 4	esPrimo Inicial	i	num%i==0	esPrimo Final	Salida
2	V	1	2 (no entra al while interno)		1	Imprime 2
3	V	1	2 (no entra al while interno)		1	Imprime 3
4	V	1	2 (Entra)	4%2==0 (V)	0	No imprime

1.10. Factorial

Análisis

Entrada: Numero entero mayor igual a 0.

Proceso: Utilizar un acumulador de multiplicación que inicie en 1. Con un ciclo, multiplicar el acumulador por cada número desde 1 hasta n.

Salida: El cálculo de factorial n!.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Numero Ingresado	n	Int	0
Contador	i	Int	1
Acumulador mult.	fact	Int	1

Seudocódigo

```
Algoritmo Problema_1_10
   Definir n, i, fact Como Entero
   i <- 1
   fact <- 1
   Escribir "Ingrese un numero entero (>= 0):"
   Leer n
   Si n >= 0 Entonces
      Mientras i <= n Hacer
         fact <- fact * i
         i <- i + 1
      FinMientras
      Escribir "El factorial es: ", fact
   SiNo
      Escribir "El factorial no esta definido para numeros negativos."
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>
```

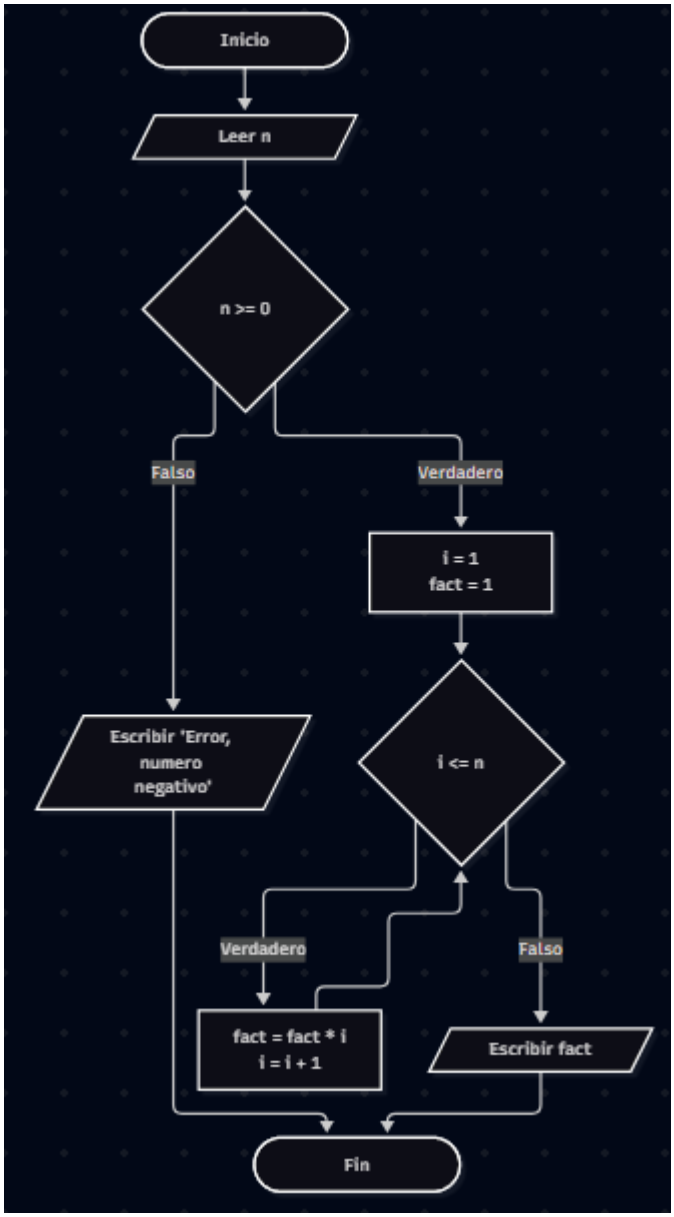
```
int main() {
    int n, i = 1, fact = 1;

    printf("Ingrese un numero entero (>= 0): ");
    scanf("%d", &n);

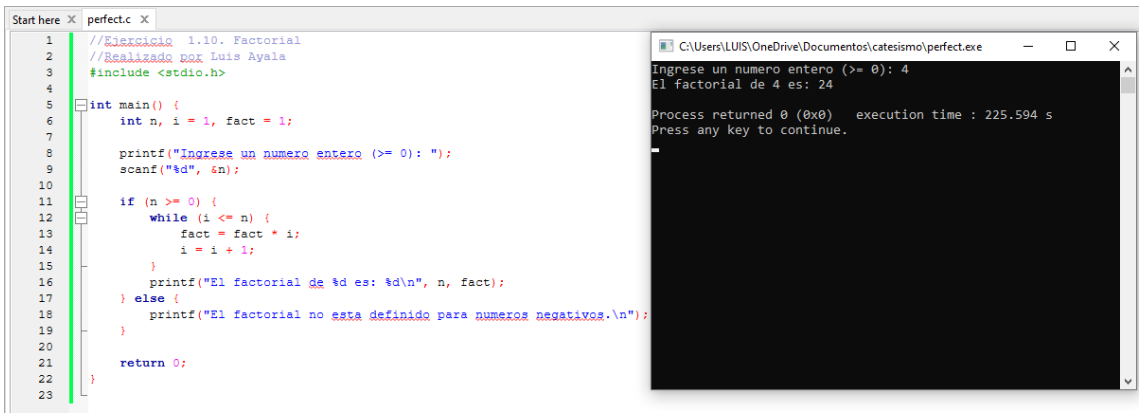
    if (n >= 0) {
        while (i <= n) {
            fact = fact * i;
            i = i + 1;
        }
        printf("El factorial de %d es: %d\n", n, fact);
    } else {
        printf("El factorial no esta definido para numeros negativos.\n");
    }

    return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



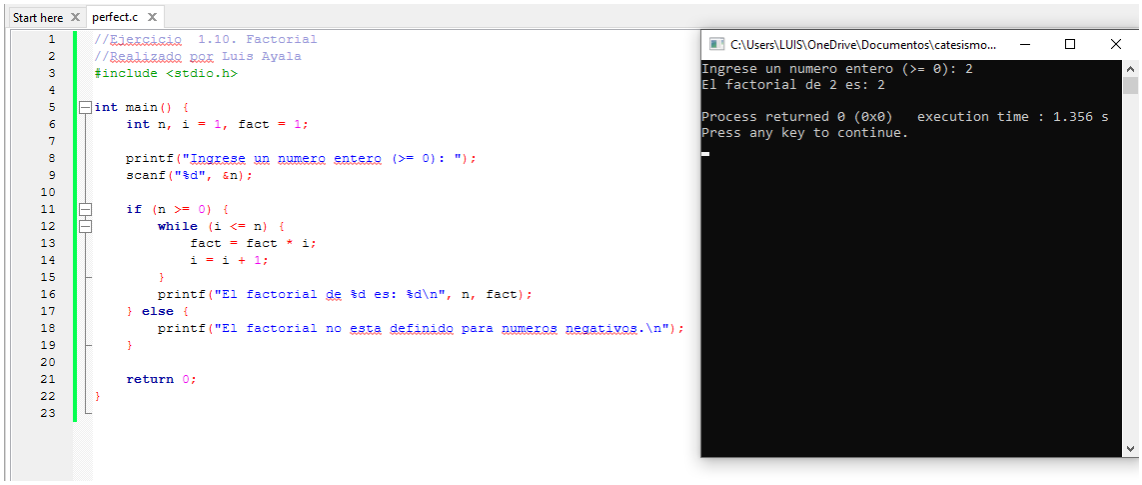
Start here | perfect.c

```
1  //Ejercicio  1.10. Factorial
2  //Realizado por Luis Ayala
3  #include <stdio.h>
4
5  int main() {
6      int n, i = 1, fact = 1;
7
8      printf("Ingrese un numero entero (>= 0): ");
9      scanf("%d", &n);
10
11     if (n >= 0) {
12         while (i <= n) {
13             fact = fact * i;
14             i = i + 1;
15         }
16         printf("El factorial de %d es: %d\n", n, fact);
17     } else {
18         printf("El factorial no esta definido para numeros negativos.\n");
19     }
20
21     return 0;
22 }
23
```

C:\Users\LUIS\OneDrive\Documentos\catesismo\perfect.exe

```
Ingrese un numero entero (>= 0): 4
El factorial de 4 es: 24

Process returned 0 (0x0)   execution time : 225.594 s
Press any key to continue.
```



Iteración	i	i<=4	fact=fact*i	i=i+1
1	1	V	1*1=1	2
2	2	V	1*2=1	3
3	3	V	2*3=6	4
4	4	V	6*4=24	5

Para Pensar

¿Qué ocurre si el usuario ingresa un número negativo?

El programa (tal como lo diseñamos con el if (n >= 0)) mostrará un mensaje de error y no calculará nada, porque la factorial matemática no está definida para negativos.

¿Y con n=20 en una variable de tipo int?

Ocurre un error de "desbordamiento" (overflow). Una variable int estándar soporta números hasta aproximadamente 2.14 mil millones. El factorial de 20 (20!) es un número gigantesco (más de 2.4 trillones), por lo que no cabe en un int y el programa arrojará un resultado basura o negativo. Se necesitaría un tipo unsigned long long.

1.11. Serie de Fibonacci

Análisis

Entrada: Numero entero mayor igual a 1 que indica cuantos terminosde las serie queremos ver.

Proceso: La serie arranca con 0 y 1. Cada numero nuevo es la suma de los dos anteriores. Se usa un ciclo de n vueltas. En cada vuelta imprimiremos el termino actual, sumamos para sacar el siguiente, y "desplazamos" los valores.

Salida: Los primeros n números de la secuencia en pantalla.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Límite de Términos	n	Int	0
Termino anterior	a	Int	0
Termino Actual	b	Int	1
Siguiente termino (Auxiliar)	c	Int	0
Contador de Vueltas	i	Int	1

Seudocódigo

```
Algoritmo Problema_1_11
   Definir n, a, b, c, i Como Entero
   a <- 0
   b <- 1
   i <- 1
   Escribir "Ingrese cuantos terminos de Fibonacci desea:"
   Leer n
   Si n >= 1 Entonces
      Escribir "Serie:"
      Mientras i <= n Hacer
```

```
        Escribir a
        c <- a + b
        a <- b
        b <- c
        i <- i + 1
    FinMientras
  SiNo
    Escribir "Error: Ingrese un numero mayor o igual a 1."
  FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

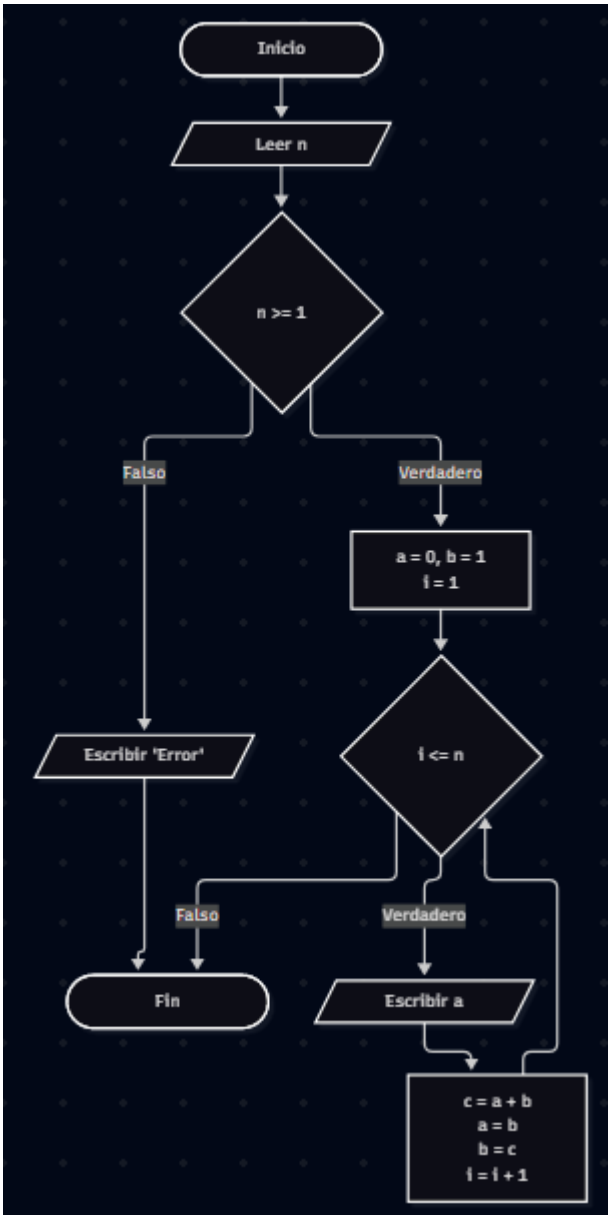
int main() {
  int n, a = 0, b = 1, c, i = 1;

  printf("Ingrese cuantos terminos de Fibonacci desea: ");
  scanf("%d", &n);

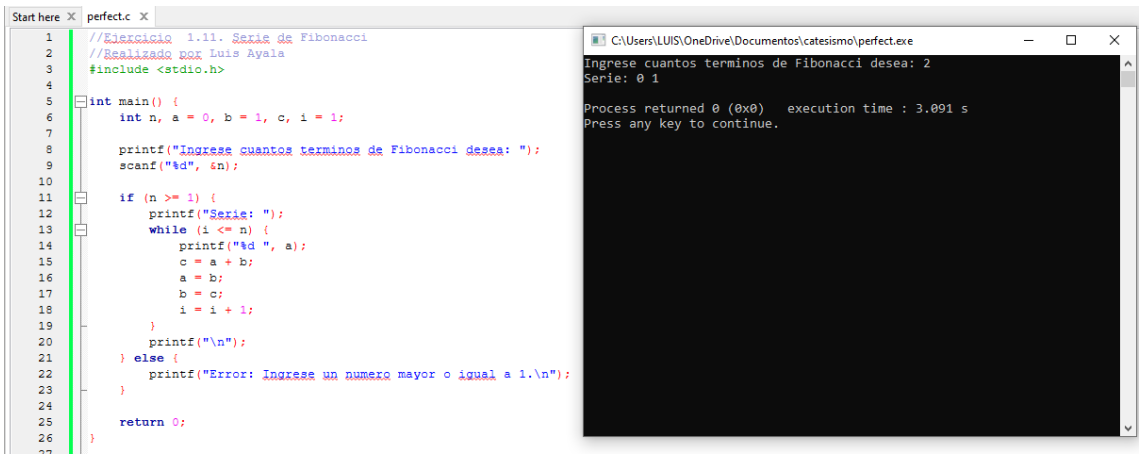
  if (n >= 1) {
    printf("Serie: ");
    while (i <= n) {
      printf("%d ", a);
      c = a + b;
      a = b;
      b = c;
      i = i + 1;
    }
    printf("\n");
  } else {
    printf("Error: Ingrese un numero mayor o igual a 1.\n");
  }

  return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



```
Start here | perfect.c

1  //Ejercicio  1.11. Serie de Fibonacci
2  //Realizado por Luis Ayala
3  #include <stdio.h>
4
5  int main() {
6      int n, a = 0, b = 1, c, i = 1;
7
8      printf("Ingrese cuantos terminos de Fibonacci desea: ");
9      scanf("%d", &n);
10
11     if (n >= 1) {
12         printf("Serie: ");
13         while (i <= n) {
14             printf("%d ", a);
15             c = a + b;
16             a = b;
17             b = c;
18             i = i + 1;
19         }
20         printf("\n");
21     } else {
22         printf("Error: Ingrese un numero mayor o igual a 1.\n");
23     }
24
25     return 0;
26 }
```

```
C:\Users\LUIS\OneDrive\Documentos\catesismo\perfect.exe
Ingrese cuantos terminos de Fibonacci desea: 2
Serie: 0 1

Process returned 0 (0x0)   execution time : 3.091 s
Press any key to continue.
```

i	i<=4	a(Salida)	c=a+b	a(Nuevo)	b(Nuevo)	i(Nuevo)
1	V	0	0+1=1	1	1	2
2	V	1	1+1=2	1	2	3
3	V	1	1+2=3	2	3	4
4	V	2	2+3=5	3	5	5(Sale)

Para Pensar

¿Cómo cambia tu algoritmo si solo debes mostrar el término número n y no toda la serie?

Cambia la ubicación de la impresión. En lugar de tener el printf dentro del bucle while para que imprima en cada vuelta, se quita de ahí y se coloca un único printf fuera del bucle, justo después de que este termine, para imprimir únicamente el valor final calculado.

1.12. Suma de dígitos

Análisis

Entrada: Un numero entero Positivo.

Proceso: Se utiliza el módulo (% 10) para extraer el último dígito y se suma a un acumulador. Luego, se utiliza la división entera (/ 10) para eliminar ese dígito del número. Esto se repite mientras el número sea mayor a 0.

Salida: La suma de los dígitos.

Tabla de Objetos

Objeto	Nombre	Tipo de Dato	Valor Inicial
Numero ingresado	num	Int	0

Acumulador de Suma	suma	Int	0
Digito Extraído	digito	Int	0

Seudocódigo

```
Algoritmo Problema_1_12
   Definir num, suma, digito Como Entero
   suma <- 0
   Escribir "Ingrese un numero entero positivo:"
   Leer num
   Si num > 0 Entonces
      Mientras num > 0 Hacer
         digito <- num MOD 10
         suma <- suma + digito
         num <- TRUNC(num / 10)
      FinMientras
      Escribir "La suma de los digitos es: ", suma
   SiNo
      Escribir "Error: Ingrese un numero positivo."
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
   int num, suma = 0, digito;

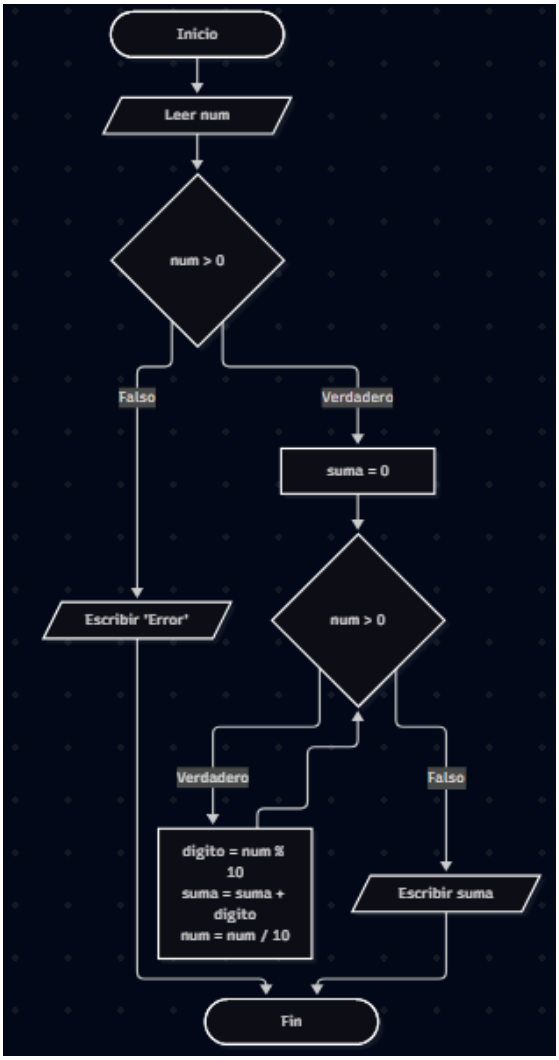
   printf("Ingrese un numero entero positivo: ");
   scanf("%d", &num);

   if (num > 0) {
      while (num > 0) {
         digito = num % 10;
         suma = suma + digito;
         num = num / 10;
      }
      printf("La suma de los digitos es: %d\n", suma);
   } else {
      printf("Error: Ingrese un numero positivo.\n");
   }

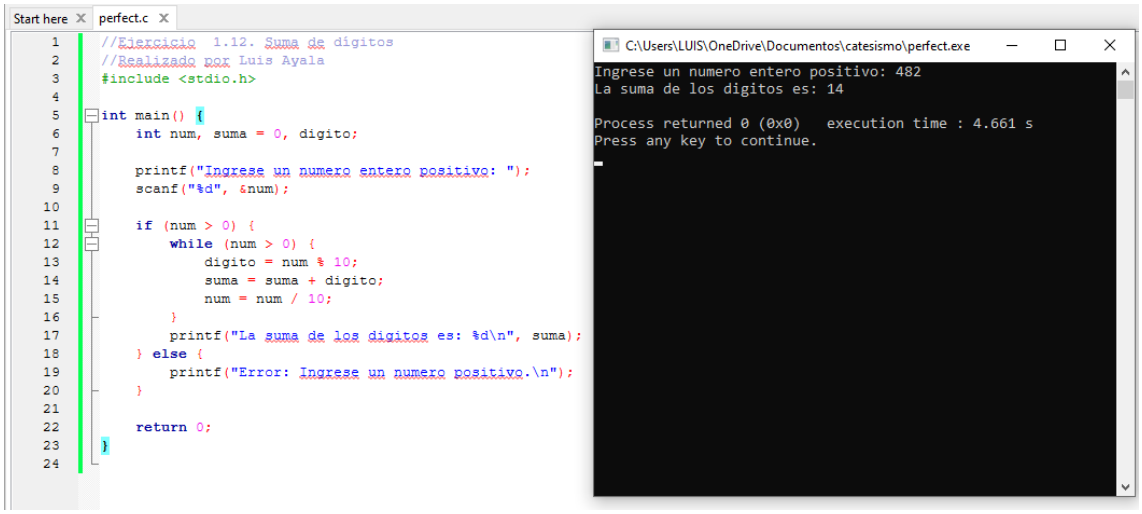
   return 0;
```

}

Diagrama de Flujo



Prueba de Escritorio



Iteracion	**Condicion**	**digito = num % 10**	**suma = suma + digito**	**num = num / 10**
1	482 > 0 (V)	2	0+2=2	482/10=48
2	48 > 0 (V)	8	2+8=10	48/10=4
3	4 > 0 (V)	4	10+4=14	4/10=0
4	0 > 0 (F -Sale)			

Para Pensar

¿Cuántas veces se repite tu ciclo para un número de k dígitos?

Justifica tu respuesta. Se repite exactamente k veces. Justificación: En cada iteración del ciclo, la operación num = num / 10 elimina exactamente el último dígito de la derecha (por ser división entera). Por lo tanto, si el número tiene 4 dígitos (k=4), tomará exactamente 4 divisiones sucesivas para que el número llegue a 0 y el ciclo se detenga.

1.13. Número capicúa

Análisis

Entrada: Un numero entero Positivo.

Proceso: Extraemos los dígitos de derecha a izquierda y lo armamos en otra variable (multiplicando por 10 y sumando el digito). Necesitamos guardar el numero original en una variable auxiliar para poder compararlo al final.

Salida: Confirmar si el numero original es el numero invertido.

Tabla de objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Número a Evaluar	num	Int	0
Copia del Original	original	Int	0
Número Invertido	invertido	Int	0
Dígito Extraído	digito	Int	0

Seudocódigo

```
Algoritmo Problema_1_13
   Definir num, original, invertido, digito Como Entero
   invertido <- 0
   Escribir "Ingrese un numero positivo:"
   Leer num
   Si num > 0 Entonces
      original <- num
      Mientras num > 0 Hacer
         digito <- num MOD 10
         invertido <- (invertido * 10) + digito
         num <- TRUNC(num / 10)
      FinMientras
      Si original = invertido Entonces
         Escribir "El numero SI es capicua."
      SiNo
         Escribir "El numero NO es capicua."
      FinSi
   SiNo
```

```
        Escribir "Error: Ingrese un numero positivo."
    FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
    int num, original, invertido = 0, digito;

    printf("Ingrese un numero positivo: ");
    scanf("%d", &num);

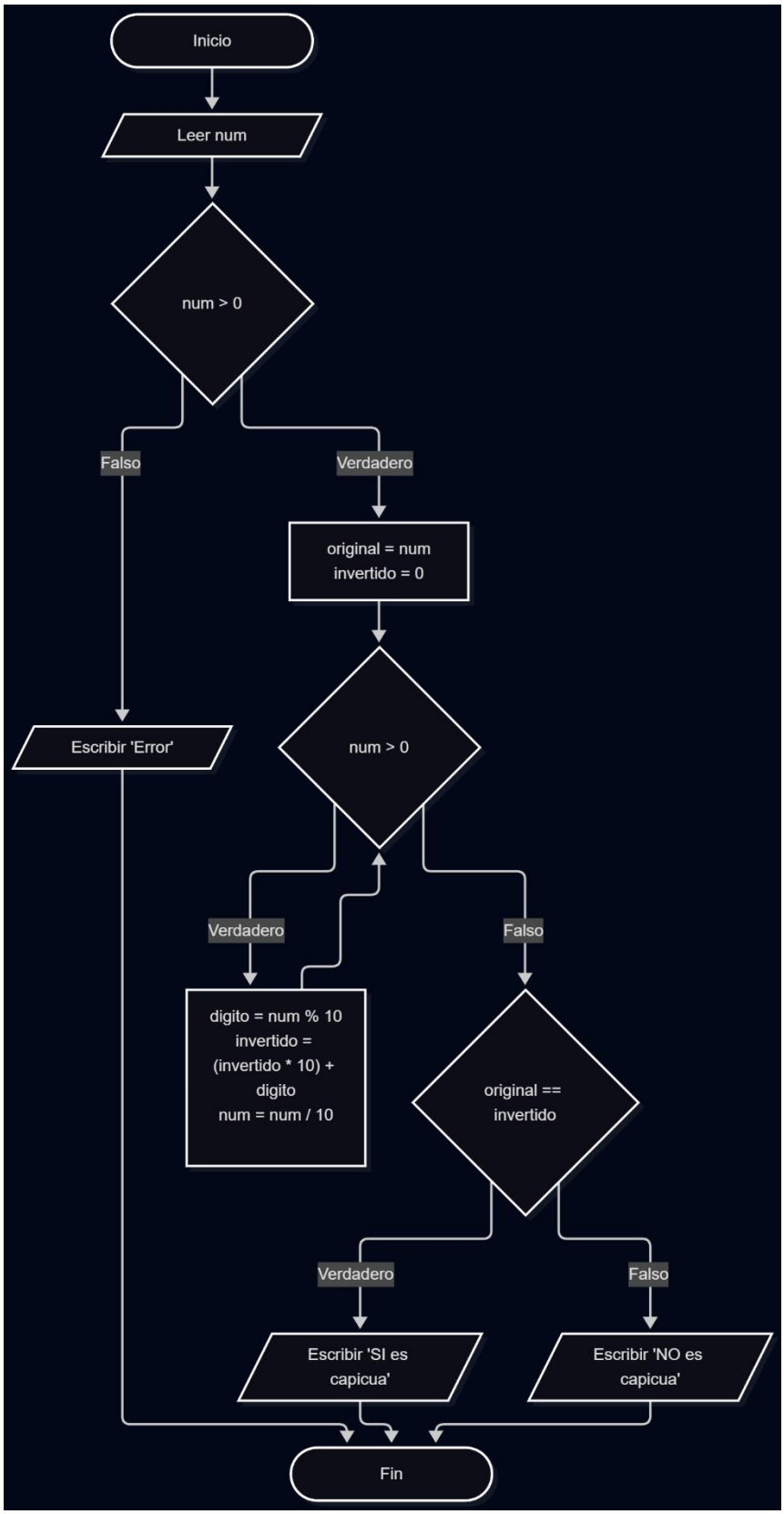
    if (num > 0) {
        original = num;

        while (num > 0) {
            digito = num % 10;
            invertido = (invertido * 10) + digito;
            num = num / 10;
        }

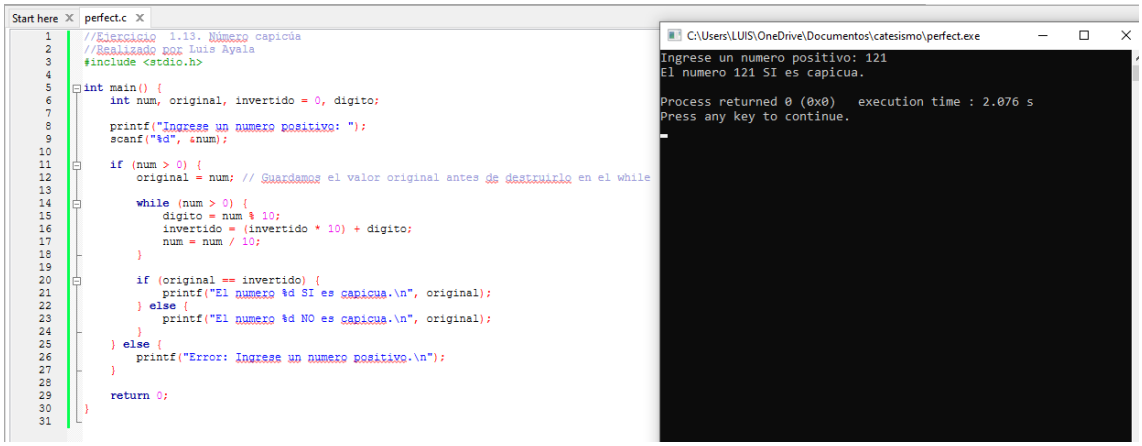
        if (original == invertido) {
            printf("El numero %d SI es capicua.\n", original);
        } else {
            printf("El numero %d NO es capicua.\n", original);
        }
    } else {
        printf("Error: Ingrese un numero positivo.\n");
    }

    return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



Iteracion	**num>0**	**digito = num % 10**	**invertido = (inv * 10) + digito**	**num = num / 10**
1	121 > 0 (V)	1	(0 * 10) + 1 = 1	12
2	12 > 0 (V)	2	(1 * 10) + 2 = 12	1
3	1 > 0 (V)	1	(12 * 10) + 1 = 121	0
4	0 > 0 (F - Sale)		¿original == invertido? 121 == 121 (Verdadero)	

Para Pensar

¿Qué casos de prueba elegirías para estar seguro de que tu solución es correcta?

Un solo dígito (ej. 7): Debe dar verdadero porque se lee igual de ambos lados.

Capicúa normal (ej. 1221): Debe dar verdadero.

No capicúa (ej. 123): Debe dar falso.

Terminado en 0 (ej. 120): Al invertir 120 matemáticamente se vuelve 21. Como 120 no es igual a 21, el programa debe dar falso y no romperse al ignorar el cero a la izquierda.

1.14. Máximo común divisor (algoritmo de Euclides)

Análisis

Entrada: Dos numero enteros positivos. Los cuales serán a y b.

Proceso: El algoritmo de Euclides establece que, mientras b no sea cero, debemos calcular el residuo de a % b. Después reemplazamos el valor de a por b, y b toma el valor del residuo.

Salida: Al finalizar el ciclo (cuando b es cero), el MCD queda guardado en la variable a.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Primero Número	a	Int	0
Segundo Número	b	Int	0
Auxiliar Residuo	resto	Int	0

Seudocódigo

```
Algoritmo Problema_1_14
   Definir a, b, resto Como Entero
   Escribir "Ingrese el primer numero positivo:"
```

```
   Leer a
   Escribir "Ingrese el segundo numero positivo:"
   Leer b
   Si a > 0 Y b > 0 Entonces
      Mientras b <> 0 Hacer
         resto <- a MOD b
         a <- b
         b <- resto
      FinMientras
      Escribir "El Maximo Comun Divisor (MCD) es: ", a
   SiNo
      Escribir "Error: Ambos numeros deben ser positivos."
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

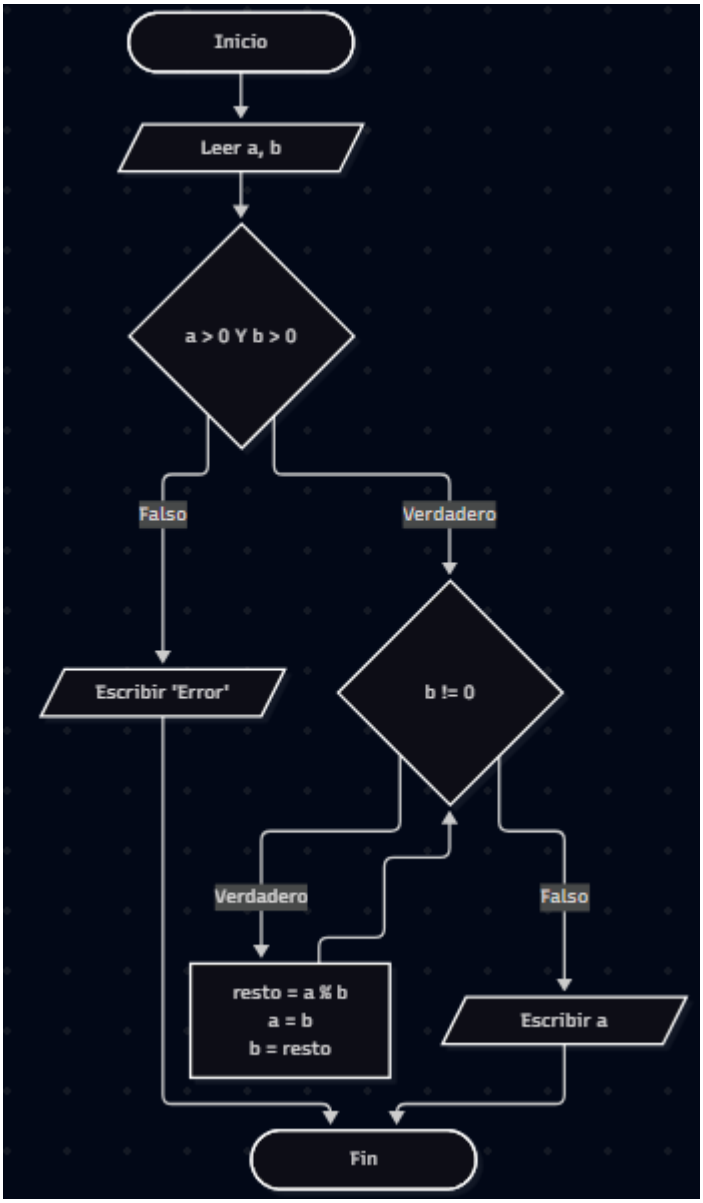
int main() {
   int a, b, resto;

   printf("Ingrese el primer numero positivo: ");
   scanf("%d", &a);
   printf("Ingrese el segundo numero positivo: ");
   scanf("%d", &b);

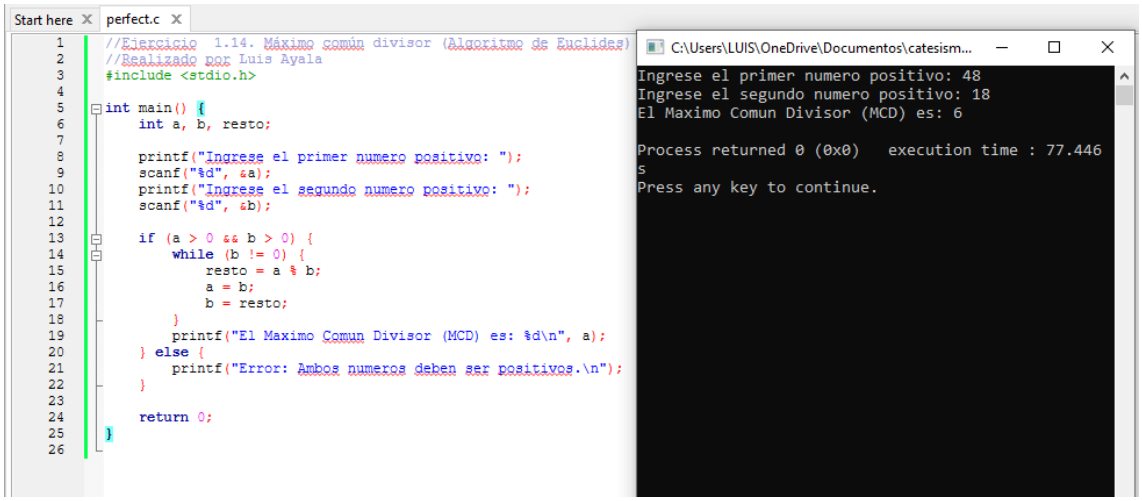
   if (a > 0 && b > 0) {
      while (b != 0) {
         resto = a % b;
         a = b;
         b = resto;
      }
      printf("El Maximo Comun Divisor (MCD) es: %d\n", a);
   } else {
      printf("Error: Ambos numeros deben ser positivos.\n");
   }

   return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



Iteracion	Condicion	resto = a % b	Nuevo a (toma el valor de b)	Nuevo b (toma el valor de resto)
1	18 != 0 (V)	48 % 18 = 12	18	12
2	12 != 0 (V)	18 % 12 = 6	12	6
3	6 != 0 (V)	12 % 6 = 0	6	0
4	0 != 0 (F - Sale)			

Para Pensar

Compara este método con probar todos los divisores desde 1 hasta el menor de los dos números. **¿Cuál hace menos operaciones y por qué?**

El algoritmo de Euclides hace muchísimas menos operaciones. Justificación: Probar uno por uno los divisores de 48 y 18 tomaría 18 iteraciones del bucle. Euclides, al usar el residuo de la división (48 % 18 = 12, luego 18 % 12 = 6, luego 12 % 6 = 0), llega a la respuesta en solo 3 pasos porque da "saltos" grandes reduciendo los números proporcionalmente en lugar de restar de uno en uno.

1.15. Año bisiesto

Análisis

Entrada: Un numero entero que representara un año.

Proceso: Se debe evaluar una condición lógica combinada. Un año es bisiesto si es múltiplo de 4 (anio % 4 == 0) Y no es múltiplo de 100 (anio % 100 != 0), O en su defecto, si es múltiplo de 400 (anio % 400 == 0). (Escribimos anio ya que el programa falla con la ñ).

Salida: Un mensaje confirmando si es o no es año bisiesto.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Año a Evaluar	anio	Int	0

Seudocódigo

```
Algoritmo Problema_1_15
   Definir anio Como Entero
   Escribir "Ingrese un anio para evaluar:"
   Leer anio
   Si anio > 0 Entonces
      Si (anio MOD 4 = 0 Y anio MOD 100 <> 0) O (anio MOD 400 = 0)
Entonces
         Escribir "El anio SI es bisiesto."
      SiNo
         Escribir "El anio NO es bisiesto."
      FinSi
```

```
   SiNo
      Escribir "Error: Ingrese un anio valido mayor a 0."
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

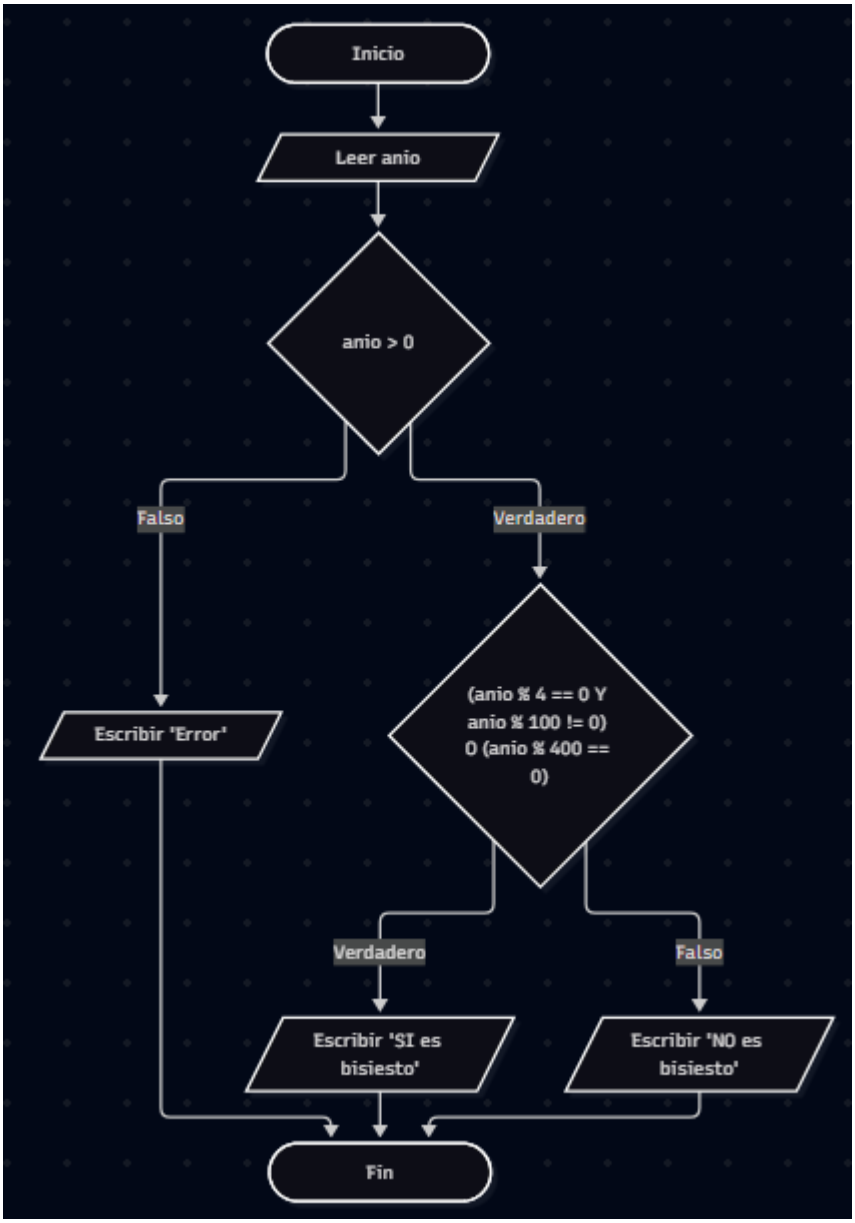
int main() {
   int anio;

   printf("Ingrese un anio para evaluar: ");
   scanf("%d", &anio);

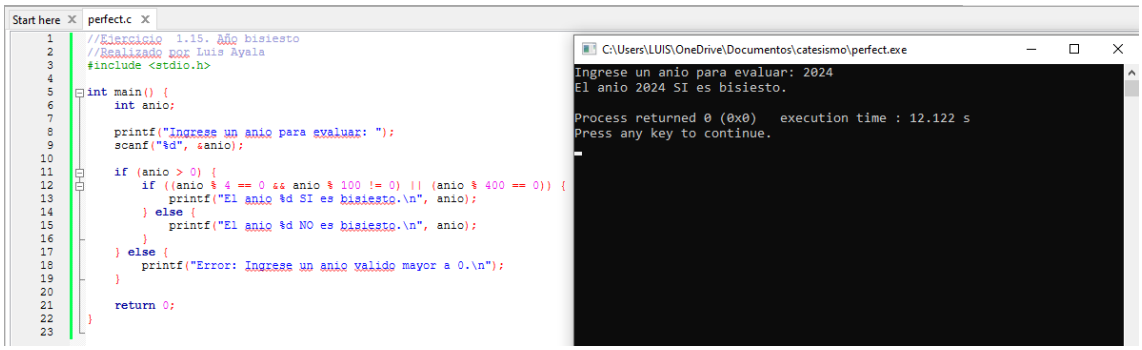
   if (anio > 0) {
      if ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)) {
         printf("El anio %d SI es bisiesto.\n", anio);
      } else {
         printf("El anio %d NO es bisiesto.\n", anio);
      }
   } else {
      printf("Error: Ingrese un anio valido mayor a 0.\n");
   }

   return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



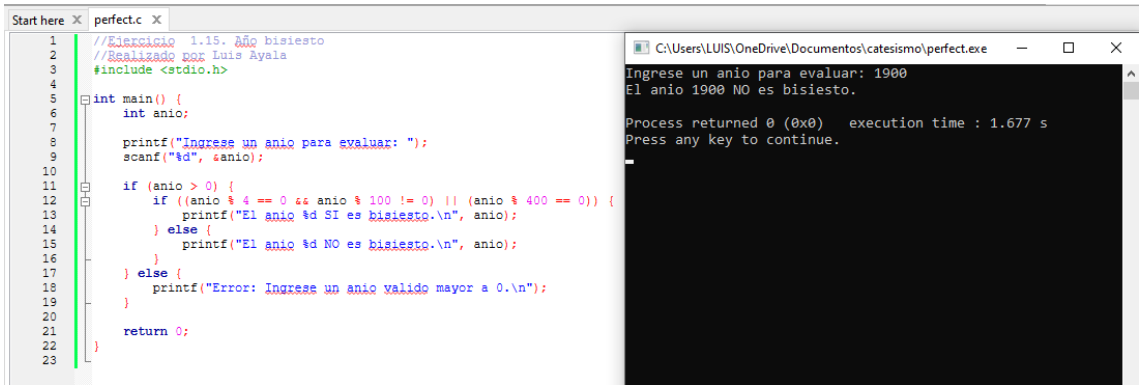
Start here | perfect.c

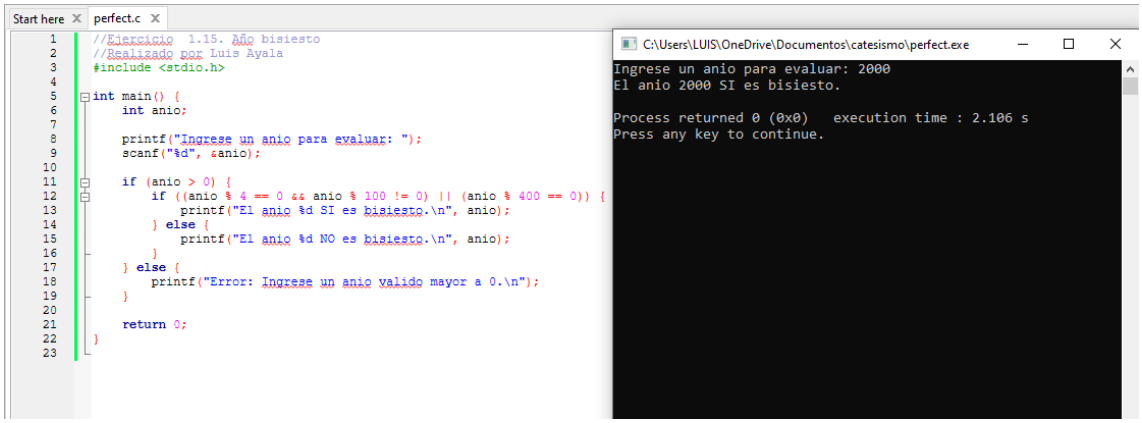
```
1  //Ejercicio  1.15. Año bisiesto
2  //Realizado por Luis Ayala
3  #include <stdio.h>
4
5  int main() {
6      int anio;
7
8      printf("Ingrese un anio para evaluar: ");
9      scanf("%d", &anio);
10
11     if (anio > 0) {
12         if ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)) {
13             printf("El anio %d SI es bisiesto.\n", anio);
14         } else {
15             printf("El anio %d NO es bisiesto.\n", anio);
16         }
17     } else {
18         printf("Error: Ingrese un anio valido mayor a 0.\n");
19     }
20
21     return 0;
22 }
23
```

C:\Users\LUIS\OneDrive\Documentos\catesismo\perfect.exe

```
Ingrese un anio para evaluar: 2024
El anio 2024 SI es bisiesto.

Process returned 0 (0x0)   execution time : 12.122 s
Press any key to continue.
```





anio	**anio % 4 == 0**	**anio % 100 != 0**	**anio % 400 == 0**	**Evaluación Completa (A Y B) O C**	**Salida**
2024	2024 % 4 = 0 (V)	2024 % 100 = 24 (V)	2024 % 400 = 24 (F)	(V Y V) O F - V O F = Verdadero	El anio 2024 SI es bisiesto.
1900	1900 % 4 = 0 (V)	1900 % 100 = 0 (F)	1900 % 400 = 300 (F)	(V Y F) O F - F O F = Falso	El anio 1900 NO es bisiesto.

2000	2000 % 4 = 0 (V)	2000 % 100 = 0 (F)	2000 % 400 = 0 (V)	(V Y F) O V - F O V = Verdadero	El anio 2000 SI es bisiesto.

Para Pensar

¿Es equivalente escribir las condiciones en otro orden?

Sí, siempre y cuando se respeten los paréntesis lógicos. La condición ((anio % 4 == 0 && anio % 100 != 0) || anio % 400 == 0) es lógicamente equivalente a (anio % 400 == 0 || (anio % 4 == 0 && anio % 100 != 0)) debido a la propiedad conmutativa del operador OR (||). Probando con 1900 da Falso en ambos casos, y con 2000 da Verdadero en ambos casos.

1.16. Número perfecto

Análisis

Entrada: Un numero entero positivo.

Proceso: Un número perfecto es aquel cuya suma de divisores propios (sin incluirse a sí mismo) es igual al mismo número. Necesitamos un ciclo while que evalúe todos los números desde 1 hasta n-1. Si el número evaluado es divisor (n % i == 0), lo sumamos a un acumulador. Al final, comparamos si la suma total es igual al número original.

Salida: Un mensaje diciendo si el numero es perfecto o no.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Número a evaluar	n	Int	0

Acumulador de divisores	suma	Int	0
Iterador / Posible divisor	i	Int	1

Seudocódigo

```
Algoritmo Problema_1_16
   Definir n, suma, i Como Entero
   suma <- 0
   i <- 1
   Escribir "Ingrese un numero entero positivo:"
   Leer n
   Si n > 0 Entonces
      Mientras i < n Hacer
         Si n MOD i = 0 Entonces
            suma <- suma + i
         FinSi
         i <- i + 1
      FinMientras
      Si suma = n Entonces
         Escribir "El numero SI es perfecto."
      SiNo
         Escribir "El numero NO es perfecto."
      FinSi
   SiNo
      Escribir "Error: Ingrese un numero mayor a 0."
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
   int n, suma = 0, i = 1;

   printf("Ingrese un numero entero positivo: ");
   scanf("%d", &n);
```

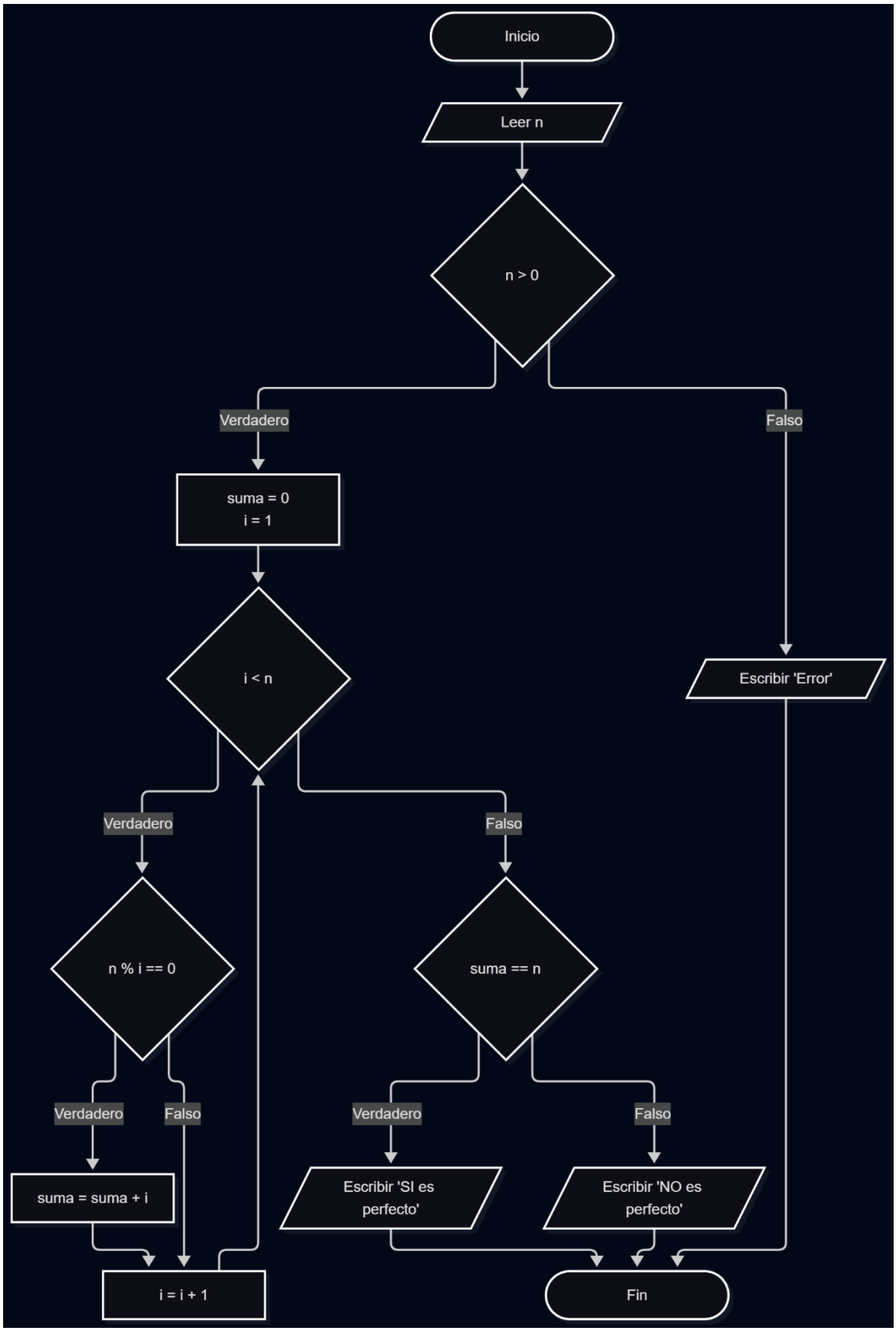
```
    if (n > 0) {

        while (i < n) {
            if (n % i == 0) {
                suma = suma + i;
            }
            i = i + 1;
        }

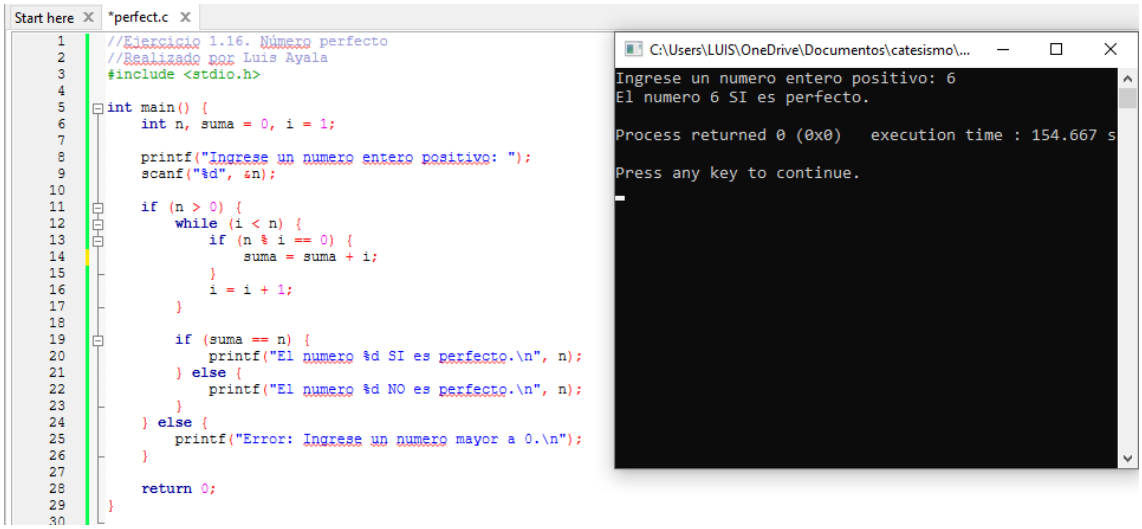

        if (suma == n) {
            printf("El numero %d SI es perfecto.\n", n);
        } else {
            printf("El numero %d NO es perfecto.\n", n);
        }
    } else {
        printf("Error: Ingrese un numero mayor a 0.\n");
    }

    return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



Iteración	**i < 6**	**n % i == 0**	**Acción si es Verdadero**	**i = i + 1**	**suma acumulada**
1	1 < 6 (V)	6 % 1 == 0 (V)	suma = 0 + 1	2	1
2	2 < 6 (V)	6 % 2 == 0 (V)	suma = 1 + 2	3	3
3	3 < 6 (V)	6 % 3 == 0 (V)	suma = 3 + 3	4	6
4	4 < 6 (V)	6 % 4 == 0 (F)		5	6
5	5 < 6 (V)	6 % 5 == 0 (F)		6	6
6	6 < 6 (F - Sale)				

Para Pensar

¿Hasta qué valor es realmente necesario recorrer los divisores? Explica cómo reducirías el número de vueltas.

Solo es necesario recorrer hasta la mitad exacta del número (n / 2).

Matemáticamente, es imposible que un número tenga un divisor entero mayor que su propia mitad (por ejemplo, para el 28, ningún número entre el 15 y el 27 puede ser divisor exacto). Reducir el ciclo hasta n / 2 corta el tiempo de ejecución a la mitad.

1.17. Conversión de decimal a binario

Análisis

Entrada: Un número entero positivo en base 10.

Proceso: Se debe dividir el número entre 2 sucesivamente para obtener los restos (0 o 1). Como los restos salen al revés, para evitar usar arreglos, armamos el número matemáticamente. Multiplicamos cada nuevo resto por un multiplicador de posición (1, 10, 100, 1000...) y lo sumamos a un acumulador. Luego actualizamos el número original dividiéndolo para 2.

Salida: La representación del número en formato binario.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Numero decimal Ingresado	n	Int	0
Resto de la Divicion	resto	Int	0

Acumulador del binario final	binario	Long Long	0
Multiplicador de Posicion	mult	Long Long	1

Seudocódigo

```
Algoritmo Problema_1_17
   Definir n, original, resto Como Entero
   Definir binario, mult Como Real
   binario <- 0
   mult <- 1
   Escribir "Ingrese un numero entero positivo:"
   Leer n
   Si n >= 0 Entonces
      Si n = 0 Entonces
         Escribir "Su representacion en binario es: 0"
      SiNo
         original <- n
         Mientras n > 0 Hacer
            resto <- n MOD 2
            binario <- binario + (resto * mult)
            n <- TRUNC(n / 2)
            mult <- mult * 10
         FinMientras
         Escribir "El numero ", original, " en binario es: ", binario
      FinSi
   SiNo
      Escribir "Error: Ingrese un numero positivo."
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
```

```
    int n, resto;
    long long binario = 0, mult = 1;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &n);

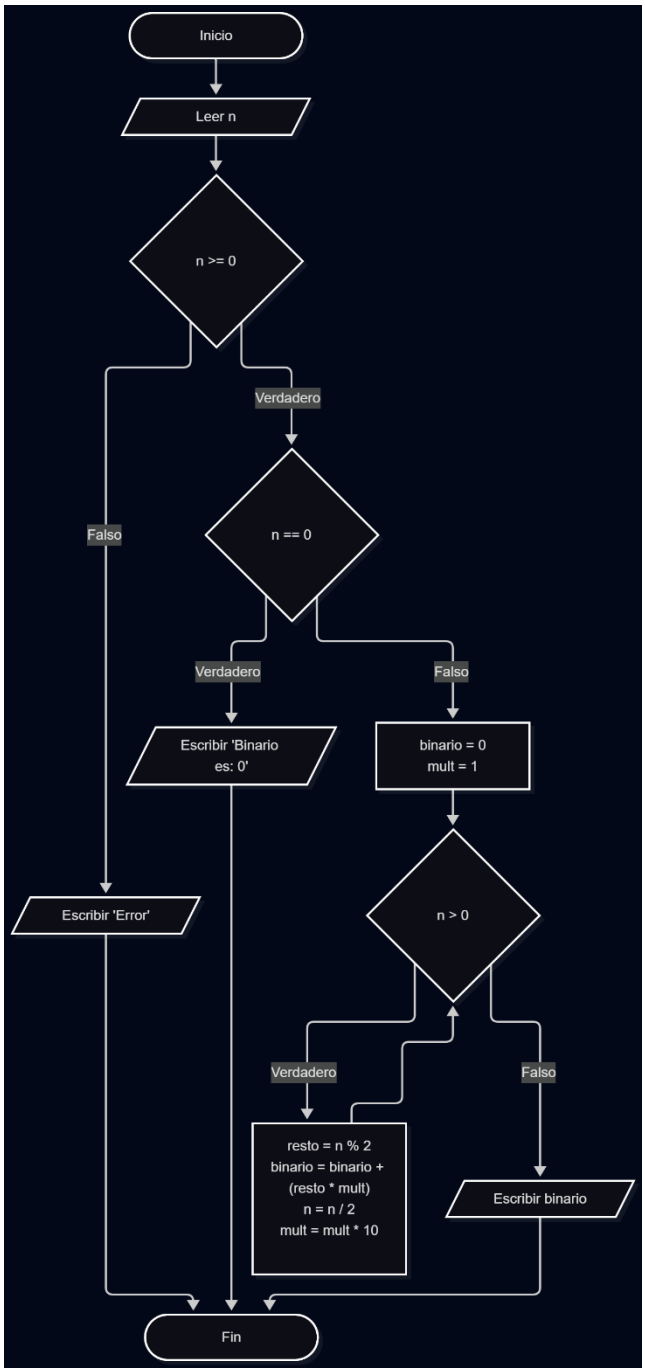
    if (n >= 0) {
        if (n == 0) {
            printf("Su representacion en binario es: 0\n");
        } else {
            int original = n;

            while (n > 0) {
                resto = n % 2;
                binario = binario + (resto * mult);
                n = n / 2;
                mult = mult * 10;
            }

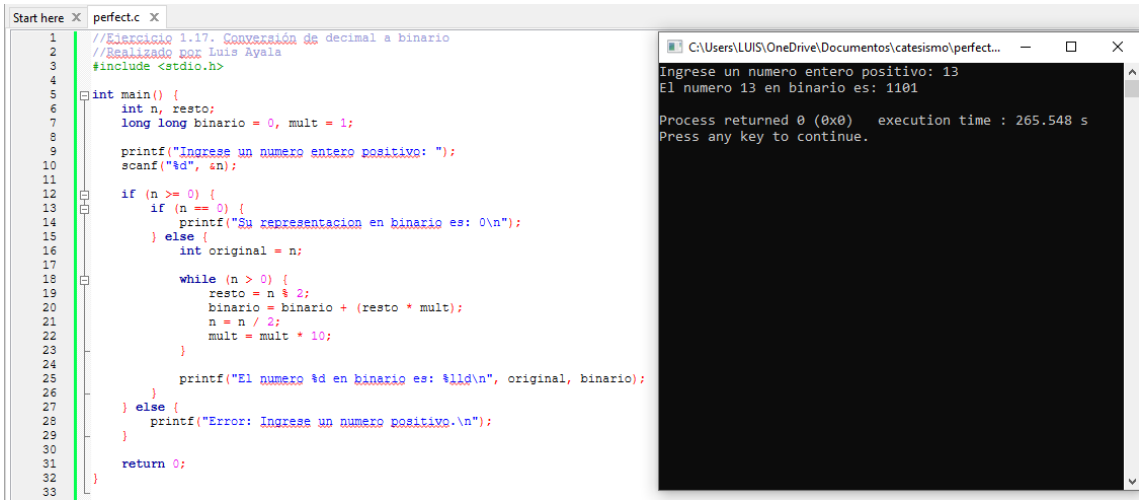
            printf("El numero %d en binario es: %lld\n", original, binario);
        }
    } else {
        printf("Error: Ingrese un numero positivo.\n");
    }

    return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



Iteración	**Condición n > 0**	**resto = n % 2**	**binario = binario + (resto * mult)**	**n = n / 2**	**mult = mult * 10**
1	13 > 0 (V)	13 % 2 = 1	0 + (1 * 1) = 1	6	10
2	6 > 0 (V)	6 % 2 = 0	1 + (0 * 10) = 1	3	100
3	3 > 0 (V)	3 % 2 = 1	1 + (1 * 100) = 101	1	1000
4	1 > 0 (V)	1 % 2 = 1	101 + (1 * 1000) = 1101	0	10000
5	0 > 0 (F - Sale)				

Para Pensar

Los restos se obtienen en orden inverso al que se leen. **¿Cómo resolverías ese detalle sin usar arreglos?**

Como aplicamos en el código, se resuelve reconstruyendo el número matemáticamente de derecha a izquierda. Se usa un multiplicador posicional (mult que inicia en 1 y se multiplica por 10 en cada vuelta: 1, 10, 100, 1000). Al hacer binario = binario + (resto * mult), cada nuevo resto se va empujando hacia la izquierda de forma automática.

1.18. Triángulo de asteriscos

Análisis

Entrada: Un número entero n>=1 que define la cantidad de filas del triángulo.

Proceso: Necesitamos usar ciclos anidados. El ciclo externo controlará en qué fila estamos (variable i). El ciclo interno se encargará de imprimir los asteriscos de esa fila (variable j). La clave está en que el ciclo interno debe dar tantas vueltas como indique la fila actual (es decir, en la fila 3, imprime 3 asteriscos). Al terminar de imprimir los asteriscos de una fila, se debe hacer un salto de línea (\n).

Salida: Un dibujo en pantalla formando un triángulo rectángulo con asteriscos.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Numero de Filas	n	Int	0
Contador de filas (externo)	i	Int	1

Contador de filas (externo)	J	Int	1

Seudocódigo

```
Algoritmo Problema_1_18
   Definir n, i, j Como Entero
   i <- 1
   Escribir "Ingrese el numero de filas (n >= 1):"
   Leer n
   Si n >= 1 Entonces
      Mientras i <= n Hacer
         j <- 1
         Mientras j <= i Hacer
            Escribir Sin Saltar "*"
            j <- j + 1
         FinMientras
         Escribir ""
         i <- i + 1
      FinMientras
   SiNo
      Escribir "Error: Ingrese un numero mayor o igual a 1."
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
   int n, i = 1, j;

   printf("Ingrese el numero de filas (n >= 1): ");
   scanf("%d", &n);
```

```
    if (n >= 1) {

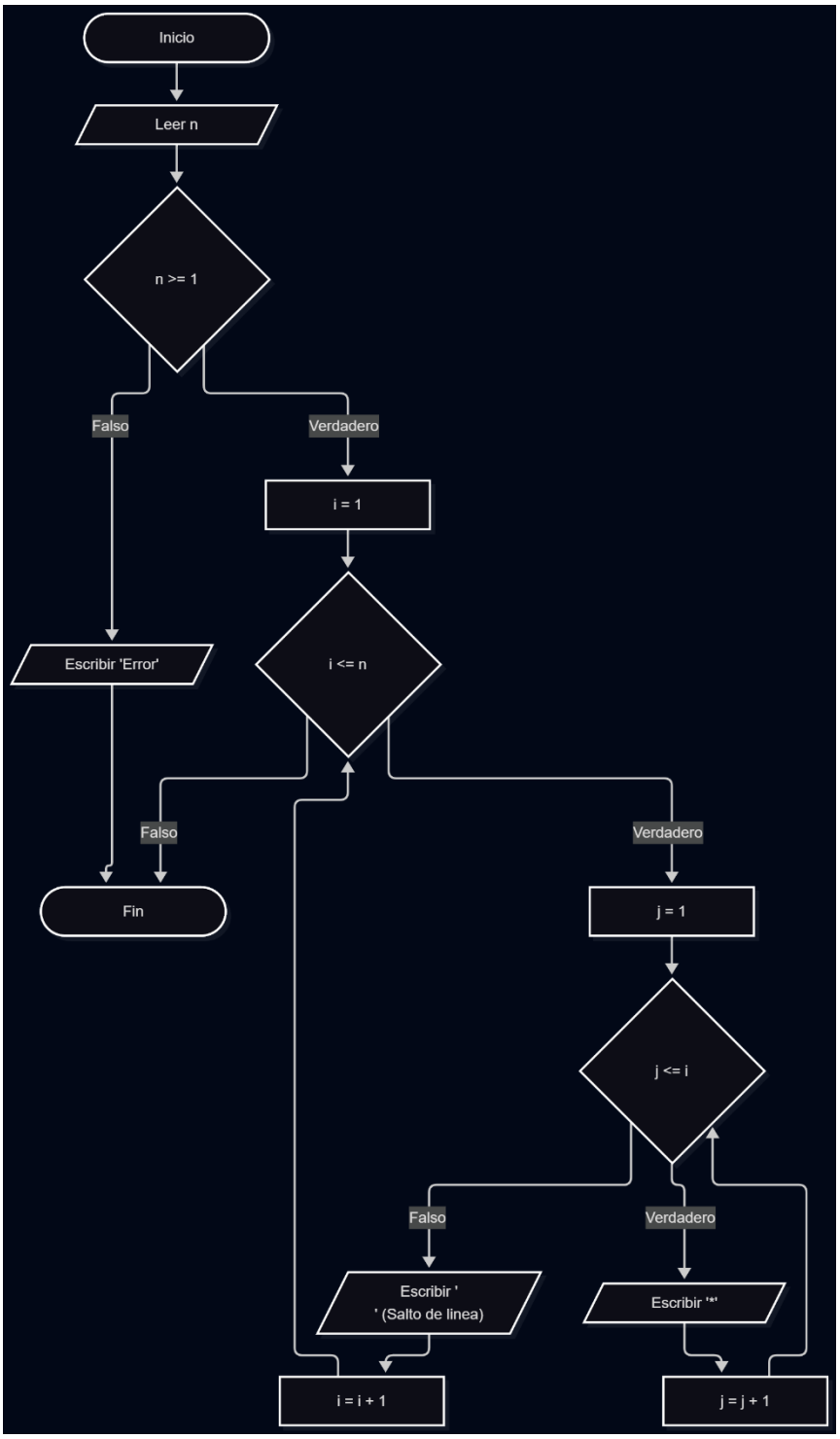
        while (i <= n) {
            j = 1;

            while (j <= i) {
                printf("*");
                j = j + 1;
            }

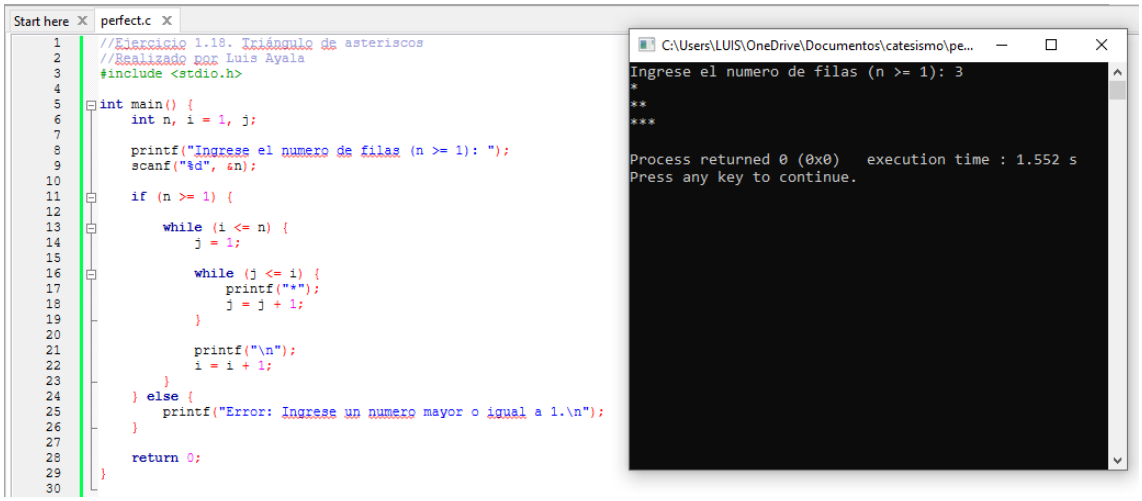
            printf("\n");
            i = i + 1;
        }
    } else {
        printf("Error: Ingrese un numero mayor o igual a 1.\n");
    }

    return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



Ciclo Externo (i)	**Condición i <= 3**	**Ciclo Interno (j)**	**Condición j <= i**	**Acción**	**Pantalla (Acumulado)**
1	V	1	1 <= 1 (V)	Imprime *, j = 2	*
		2	2 <= 1 (F-Sale)	Imprime \n, i = 2	
2	V	1	1 <= 2 (V)	Imprime *, j = 2	*
		2	2 <= 2 (V)	Imprime *, j = 3	**
		3	3 <= 2 (F-Sale)	Imprime \n, i = 3	
3	V	1	1 <= 3 (V)	Imprime *, j = 2	*
		2	2 <= 3 (V)	Imprime *, j = 3	**

		3	3 <= 3 (V)	Imprime *, j = 4	***
		4	4 <= 3 (F-Sale)	Imprime \n, i = 4	
4	F-Sale			Fin	

Para Pensar

Si ahora quisieras un triángulo centrado (una pirámide), **¿qué parte de tu algoritmo cambiaría y qué parte se mantendría?**

Se mantendría el ciclo externo que controla las filas (i). Lo que cambiaría es el contenido de cada fila: antes de imprimir los asteriscos de la fila actual, habría que agregar un nuevo ciclo interno para imprimir espacios en blanco decrecientes, y el ciclo de los asteriscos pasaría a imprimir cantidades impares (2*i - 1) en lugar de i asteriscos.

1.19. Segundo mayor de una secuencia

Análisis

Entrada: Números enteros positivos ingresados por el usuario, donde el número 0 actúa como centinela.

Proceso: Necesitamos un ciclo while que pida números mientras sean diferentes de 0. Por cada número válido, sumamos 1 a un contador. Para calcular los mayores, usamos la siguiente lógica:

Si el nuevo número es mayor que el actual campeón (mayor1), entonces el campeón antiguo es desplazado y pasa a ser el mayor2, mientras que el nuevo número se corona como mayor1.

Si el nuevo número no le gana al mayor1, pero es mayor que mayor2, entonces solo actualizamos mayor2.

Salida: Informar cuál es el mayor, cuál el segundo mayor, o dar un mensaje de error si se ingresaron menos de dos números.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Números Ingresados	num	Int	0
Primer mayor	mayor1	Int	0
Segundo Mayor	mayor2	Int	0
Contador de Números	cant	Int	0

Seudocódigo

```
Algoritmo Problema_1_19
   Definir num, mayor1, mayor2, cant Como Entero
   mayor1 <- 0
   mayor2 <- 0
   cant <- 0
   Escribir "Ingrese un numero positivo (0 para terminar):"
   Leer num
   Mientras num <> 0 Hacer
      Si num > 0 Entonces
         cant <- cant + 1
         Si num > mayor1 Entonces
            mayor2 <- mayor1
            mayor1 <- num
         SiNo
            Si num > mayor2 Y num <> mayor1 Entonces
```

```
            mayor2 <- num
          FinSi
        FinSi
      FinSi
      Escribir "Ingrese otro numero (0 para terminar):"
      Leer num
   FinMientras
   Si cant < 2 Entonces
      Escribir "Error: Hay menos de dos valores."
   SiNo
      Escribir "El mayor es: ", mayor1
      Escribir "El segundo mayor es: ", mayor2
   FinSi
FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
   int num, mayor1 = 0, mayor2 = 0, cant = 0;

   printf("Ingrese un numero positivo (0 para terminar): ");
   scanf("%d", &num);

   while (num != 0) {
      if (num > 0) {
         cant = cant + 1;

         if (num > mayor1) {

            mayor2 = mayor1;
            mayor1 = num;
         } else {

            if (num > mayor2 && num != mayor1) {
               mayor2 = num;
            }
         }
      }

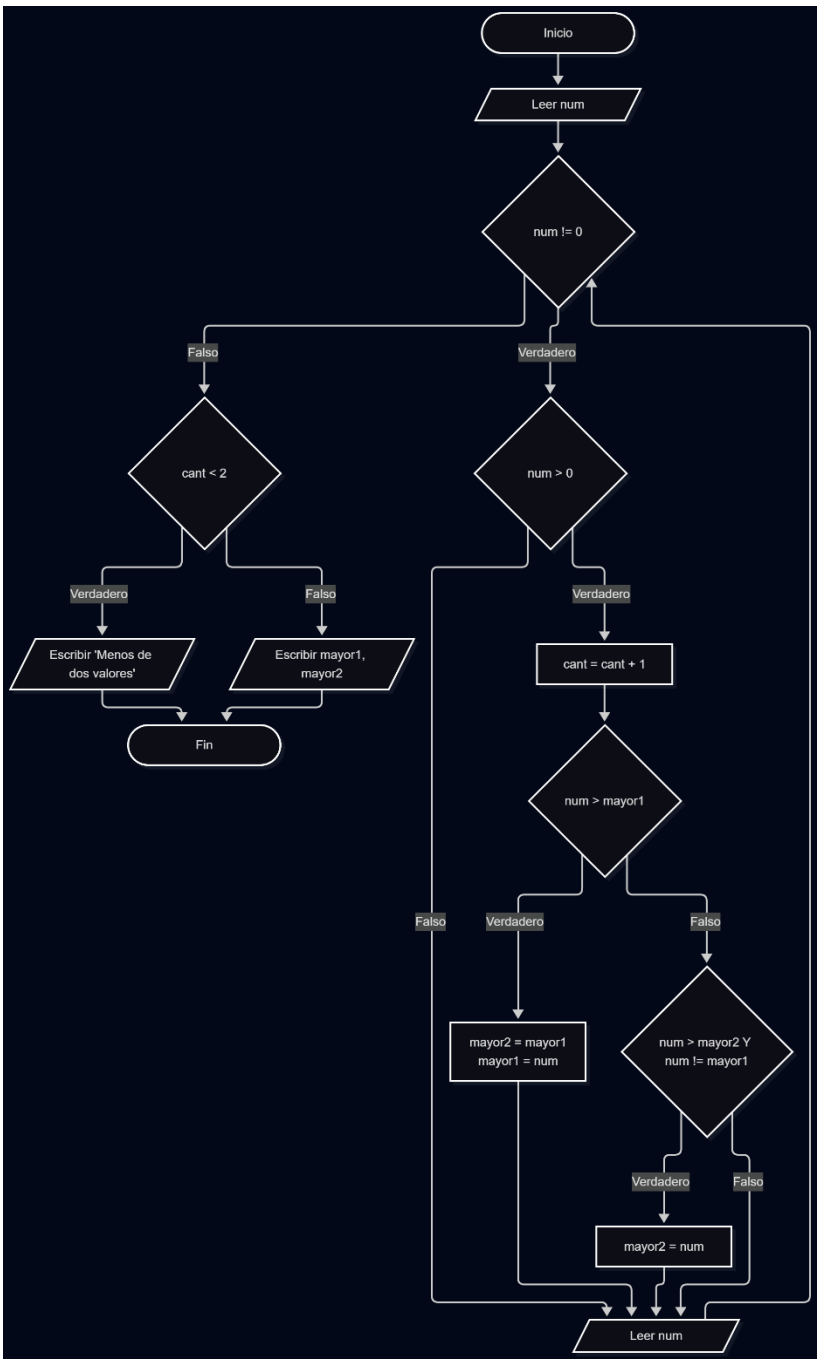
      printf("Ingrese otro numero (0 para terminar): ");
      scanf("%d", &num);
   }


   if (cant < 2) {
      printf("\nError: Hay menos de dos valores.\n");
   } else {
      printf("\nEl mayor es: %d\n", mayor1);
```

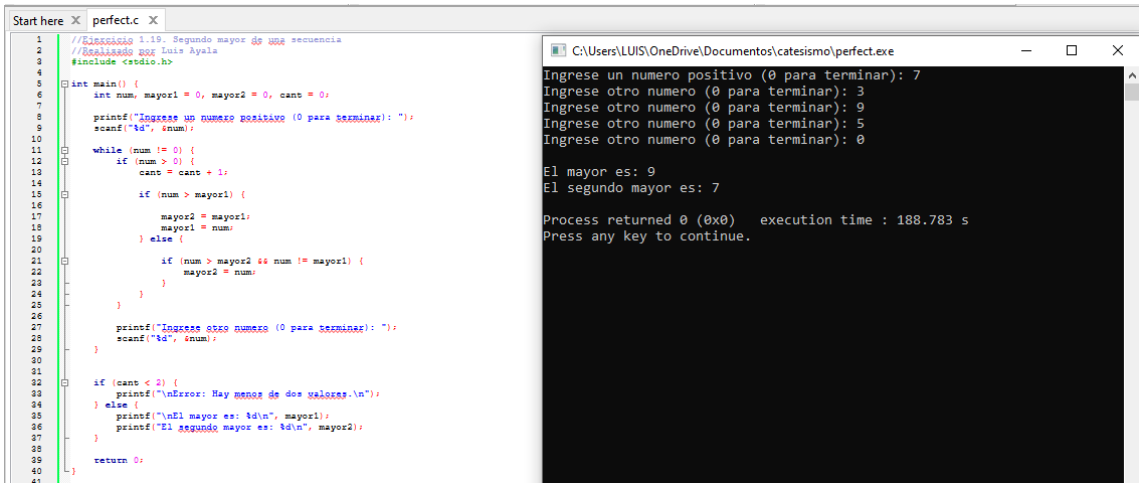
```
        printf("El segundo mayor es: %d\n", mayor2);
    }

    return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



num ingresado	**num != 0**	**num > mayor1**	**num > mayor2 && num != mayor1**	**Actualización de variables**	**cant**
7	V	7 > 0 (V)		mayor2 = 0, mayor1 = 7	1
3	V	3 > 7 (F)	3 > 0 y 3 != 7 (V)	mayor2 = 3	2
9	V	9 > 7 (V)		mayor2 = 7, mayor1 = 9	3
5	V	5 > 9 (F)	5 > 7 (F)	No cambia nada	4
0	F-Sale				4

Para Pensar

¿**Qué debería pasar si el valor mayor se repite (por ejemplo, 9, 9, 5)?** Decide, justifica y prueba tu decisión. El segundo 9 debe ser ignorado. Justificación: Si el objetivo es encontrar el "segundo mayor" numéricamente hablando en la secuencia matemática, el segundo mayor valor único en 9, 9, 5 es el 5. Por ello, en el código se añadió la condición estricta num != mayor1 para evitar que un número repetido desplace al segundo mayor legítimo.

1.20. Adivina el número

Análisis

Entrada: Los intentos del usuario (números enteros) para adivinar.

Proceso: El programa define un número secreto fijo. Usaremos un bucle while que se repetirá mientras el intento sea diferente al número secreto. En cada vuelta, el usuario ingresa un número, se suma 1 al contador de intentos, y se usan condicionales (if-else) para darle la pista de si el secreto es "más alto" o "más bajo".

Salida: Las pistas en cada intento, el mensaje de "¡acertaste!" y la cantidad total de intentos utilizados.

Tabla de Objetos

Objeto	**Nombre**	**Tipo de Dato**	**Valor Inicial**
Número a adivinar	secreto	Int	42
Número ingresado	intento	Int	0
Contador de intentos	intentos	Int	0

Seudocódigo

```
Algoritmo Problema_1_20
   Definir secreto, intento, intentos Como Entero
   secreto <- 42
   intento <- 0
   intentos <- 0
   Escribir "¡Adivina el numero secreto entre 1 y 100!"
   Mientras intento <> secreto Hacer
      Escribir "Ingresa tu intento:"
      Leer intento
      intentos <- intentos + 1
      Si intento < secreto Entonces
         Escribir "Mas alto..."
      SiNo
         Si intento > secreto Entonces
```

```
            Escribir "Mas bajo..."
         SiNo
            Escribir "¡Acertaste!"
         FinSi
      FinSi
   FinMientras
   Escribir "Te tomo ", intentos, " intentos adivinarlo."
FinAlgoritmo
```

Código

```
#include <stdio.h>

int main() {
   int secreto = 42;
   int intento = 0, intentos = 0;

   printf("¡Adivina el numero secreto entre 1 y 100!\n");

   while (intento != secreto) {
      printf("\nIngresa tu intento: ");
      scanf("%d", &intento);

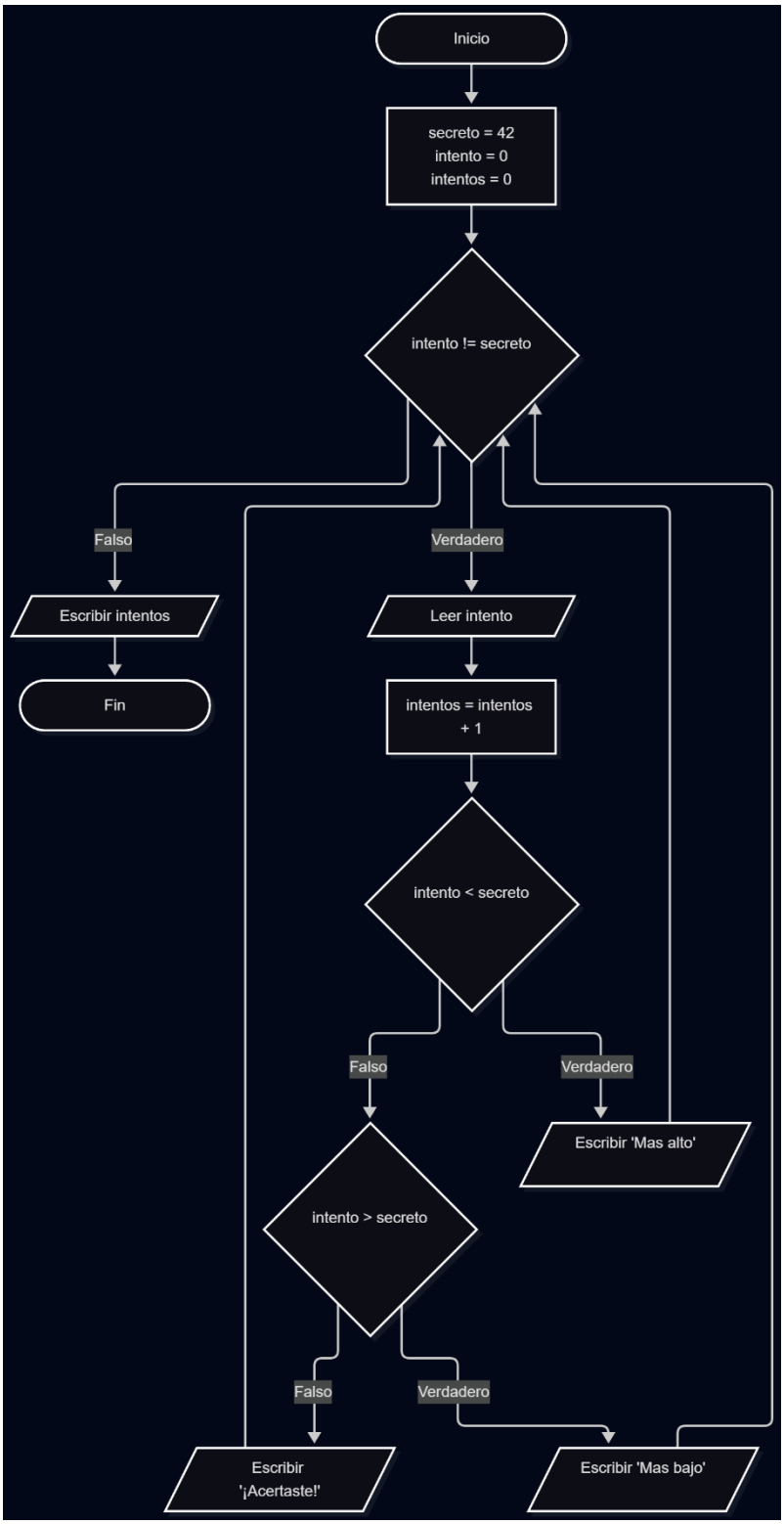
      intentos = intentos + 1;

      if (intento < secreto) {
         printf("Mas alto...\n");
      } else if (intento > secreto) {
         printf("Mas bajo...\n");
      } else {
         printf("¡Acertaste!\n");
      }
   }

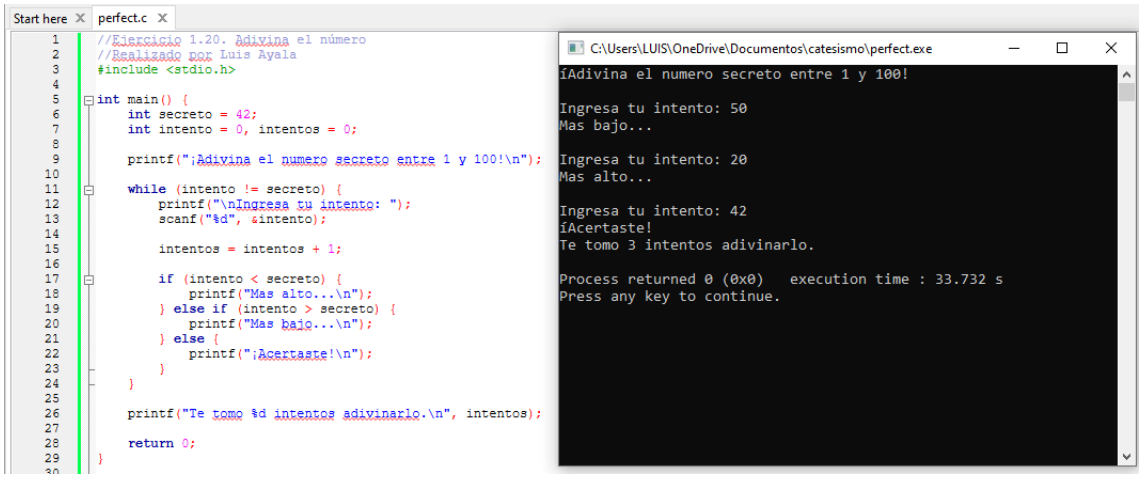
   printf("Te tomo %d intentos adivinarlo.\n", intentos);

   return 0;
}
```

Diagrama de Flujo



Prueba de Escritorio



Iteracion	Condición intento != secreto	intento (leído)	intentos acumulado	Condición 1: intento < secreto	Condición 2: intento > secreto	Mensaje en pantalla
1	0 != 42 (V)	50	1	50 < 42 (F)	50 > 42 (V)	Mas bajo...
2	50 != 42 (V)	20	2	20 < 42 (V)		Mas alto...
3	20 != 42 (V)	42	3	42 < 42 (F)	42 > 42 (F)	¡Acertaste!
4	42 != 42 (F-Sale)					Te tomo 3 intentos.

Para Pensar

Si jugaras con la mejor estrategia posible (dividir siempre el rango por la mitad), ¿cuál es el máximo de intentos que necesitarías para el rango 1-100? Justifica tu respuesta. El máximo de intentos es 7.

Justificación: La estrategia se basa en la "Búsqueda Binaria", descartando la mitad de las opciones en cada paso (100 - 50 - 25 - 12 - 6 - 3 - 1). El número máximo de divisiones para llegar a 1 equivale al logaritmo en base 2 del límite superior ($\log_2(100) \approx 6.64$), lo cual obliga a realizar un máximo de 7 intentos en el peor de los casos.

Rubrica de Pensamiento Critico

Criterio	Nivel 4 – Excelente	Nivel 3 – Satisfactorio	Nivel 2 – En desarrollo	Nivel 1 – Inicial
Comprensión y análisis del problema	Identifica con precisión entradas, salidas, restricciones y supuestos; reformula el problema con sus propias palabras.	Identifica entradas y sal correctamente; omite a supuesto o restricción meno	Identifica solo parte de las entradas o salidas; el análisis es superficial.	No distingue qué datos se piden ni qué se debe producir.
Diseño de la solución (tabla de objetos y seudocódigo)	La tabla de objetos es completa y el seudocódigo es correcto, ordenado y sin ambigüedades.	Tabla y seudocódigo co n detalles menores por m	Hay errores de lógica o faltan objetos que impiden resolver el problema por completo.	El diseño es incompleto o no resuelve el problema planteado.
Verificación y casos de prueba	La prueba de escritorio i casos normales, límite y , y detecta o corrige errores.	La prueba de escritorio es correcta con casos normales y al menos un caso límite.	La prueba de escritorio usa un solo caso o contiene errores de seguimiento.	No presenta prueba de escritorio o no se corresponde con el algoritmo.
Evaluación de alternativas y eficiencia	Compara al menos dos enfoques, argumenta cuál es mejor y justifica con datos (número de pasos, casos).	Propone una alternativa y explica de forma razonable su ventaja o desventaja.	Menciona otra alternati comparar ni justificar.	No considera alternativas a su solución.
Justificación y razonamiento	Responde las preguntas "Para pensar" con argumentos claros, coherentes y respaldados por ejemplos.	Responde con argumen correctos, aunque con profundidad o pocos ejem	Responde de forma parcial o sin sustentar la respuesta.	No responde o la respuesta es incorrecta o ajena a la pregunta.
Comunicación (diagrama de flujo y claridad)	El diagrama usa la simbología correcta, es legible y coincide totalmente con el seudocódigo.	El diagrama es correcto co errores menores de sim presentación.	El diagrama presenta errores de flujo o difiere del seudocódigo.	El diagrama falta o no se puede seguir.

Puntaje total: 18 / 24

Interpretación: 21–24 Excelente · 16–20 Satisfactorio · 11–15 En desarrollo · 6–10 Inicial

Una acción concreta para mejorar en el próximo ejercicio: Evaluar mejor los problemas para evitar errores y mejorar la eficiencia del programa.